

哈尔滨工业大学（深圳）

数据库实验指导书

实验四 数据库系统功能实现

2024 秋

目录

1. 实验目的.....	3
2. 实验环境.....	3
3. 实验任务.....	3
4. 实验步骤.....	4
4.1 环境准备	4
4.1.1 安装、导入虚拟机	4
4.1.2 使用虚拟机环境进行开发	12
4.2 开发	29
4.3 测试	34
5. 实验评分.....	37
附录 1 上传下载文件	38

1. 实验目的

1. 掌握缓冲池管理中的页面调度策略 LRU 算法。
2. 学习并实现磁盘存储管理器，缓冲池替换策略，以及缓冲池管理器。
3. 理解记录管理器的工作原理及其在数据库中的作用。
4. 掌握定长记录的组织形式和记录操作的实现。

2. 实验环境

- 操作系统：Linux
- 编程语言：C++
- 编译环境：g++
- 依赖库：Linux 标准库

3. 实验任务

本实验分为两个主要任务，分别是缓冲池管理器的实现和记录管理器的实现。每个任务又细分为若干个子任务。

任务一：缓冲池管理器

任务 1.1 磁盘存储管理器

- **目标：**实现磁盘存储管理器，负责读写指定页面、分配页面编号，以及对文件进行操作。
- **子任务：**
 1. 实现读写页面功能。
 2. 实现分配页面编号功能。
 3. 实现文件操作功能（创建、打开、关闭、删除文件）。

任务 1.2 缓冲池替换策略

- **目标：**实现缓冲池替换策略，采用最近最少使用（LRU）算法。
- **子任务：**
 1. 实现 LRU 替换策略。
 2. 实现页面固定（pin）和取消固定（unpin）功能。

任务 1.3 缓冲池管理器

- **目标：**实现缓冲池管理器，负责管理缓冲池中的页面与磁盘文件中的页面之间的来回移动。
- **子任务：**
 1. 实现新页面创建功能。
 2. 实现获取指定页面功能。
 3. 实现取消固定页面功能。
 4. 实现删除指定页面功能。
 5. 实现强制刷新页面功能。

6. 实现刷新所有页面功能。

任务二：记录管理器

任务 2.1 记录操作

- 目标：实现记录管理器，负责对文件中的记录进行操作。
- 子任务：
 1. 实现获取记录功能。
 2. 实现插入记录功能。
 3. 实现删除记录功能。
 4. 实现更新记录功能。

任务 2.2 记录迭代器

- 目标：实现记录迭代器，用于遍历文件中存放的记录。
- 子任务：
 1. 实现迭代器初始化功能。
 2. 实现找到下一个记录位置功能。
 3. 实现判断是否到达文件末尾功能。
 4. 实现获取当前记录位置功能。

4. 实验步骤

4.1 环境准备

本实验需要在 Linux 系统上运行 Rucbase 代码，我们可以使用虚拟机、Docker，或者直接安装 Ubuntu 系统。请参考官网资料：[Rucbase 环境配置文档](#)及 [Rucbase 使用文档](#)

实验室已安装配置好虚拟机环境，如果使用实验室电脑则跳过本节。如想在本机上使用虚拟机运行 Ubuntu 进行开发，可执行本节的操作。

4.1.1 安装、导入虚拟机

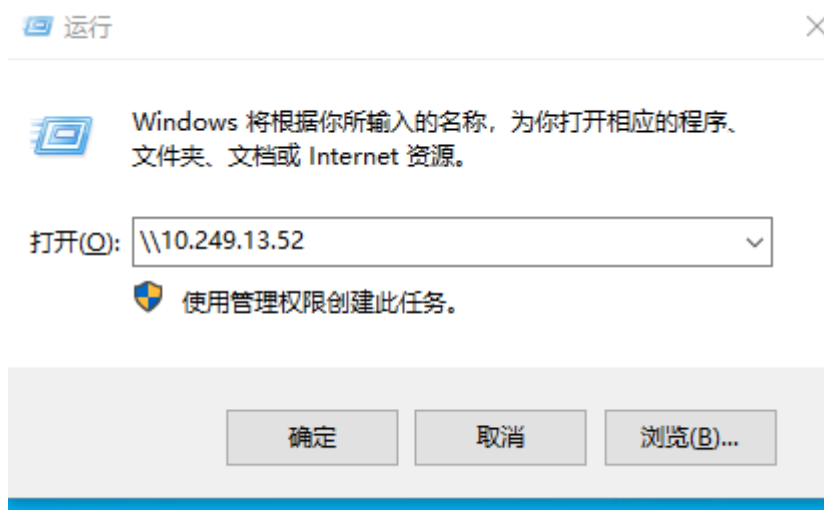
1、准备条件

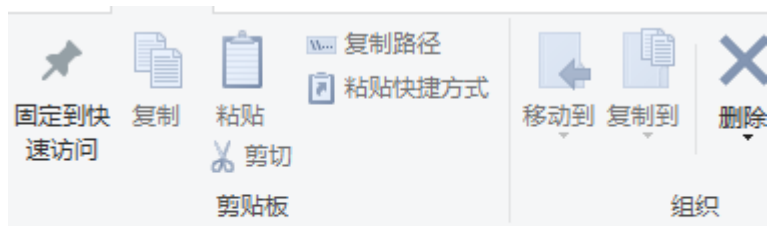
- (1) 可以通过特定的软件工具来检查虚拟化技术是否已启用。例如，在 Windows 上，可以使用任务管理器（Ctrl+Shift+Esc 打开），切换到 性能 标签，查看 CPU 的信息项中是否提及了虚拟化（例如 虚拟化：已启用）。



如果没有启动虚拟化，请根据你的电脑情况自行启动虚拟化。

(2) 获取安装软件及配置好的虚拟机：
访问：[\\10.249.13.52](http://10.249.13.52)





← → ↕ ↑ 网络 > 10.249.13.52

★ 快速访问

桌面

↓ 下载

文档

图片

此电脑

2024-09



507虚拟机

网络 > 10.249.13.52 > 507虚拟机

名称

SeedUbuntu2004.ova

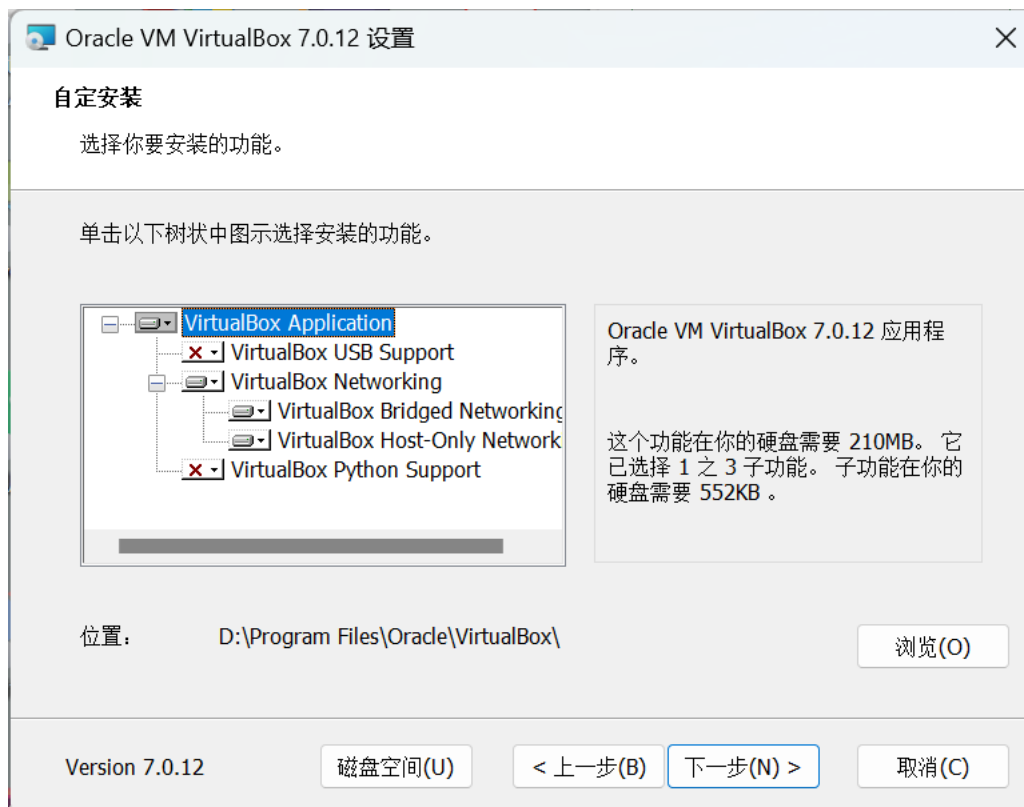
VirtualBox-7.0.12-159484-Win.exe

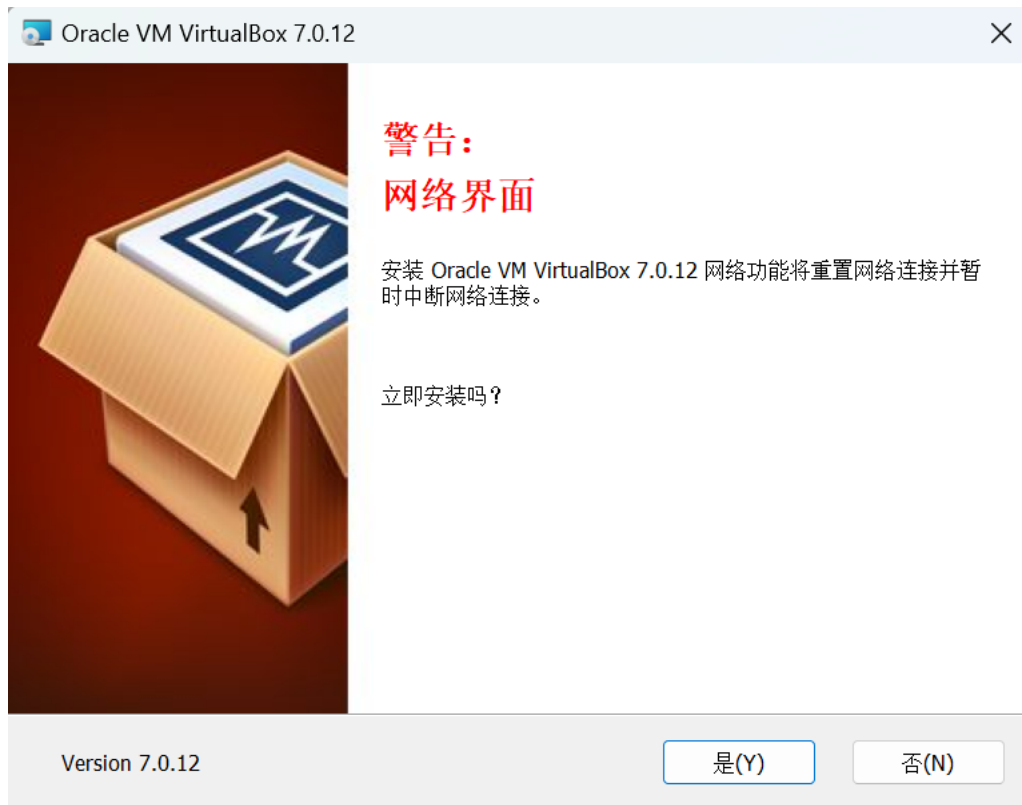
Visual Studio Code2024(64bit).zip

2、安装 Oracle VM VirtualBox

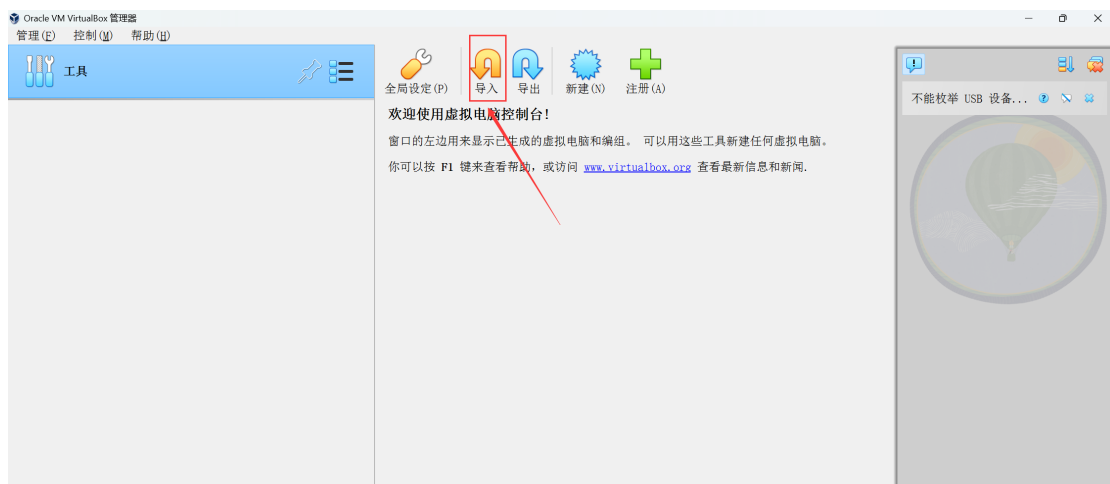


组件需至少安装：**VirtualBox Networking**，其它 2 个可以不安装：



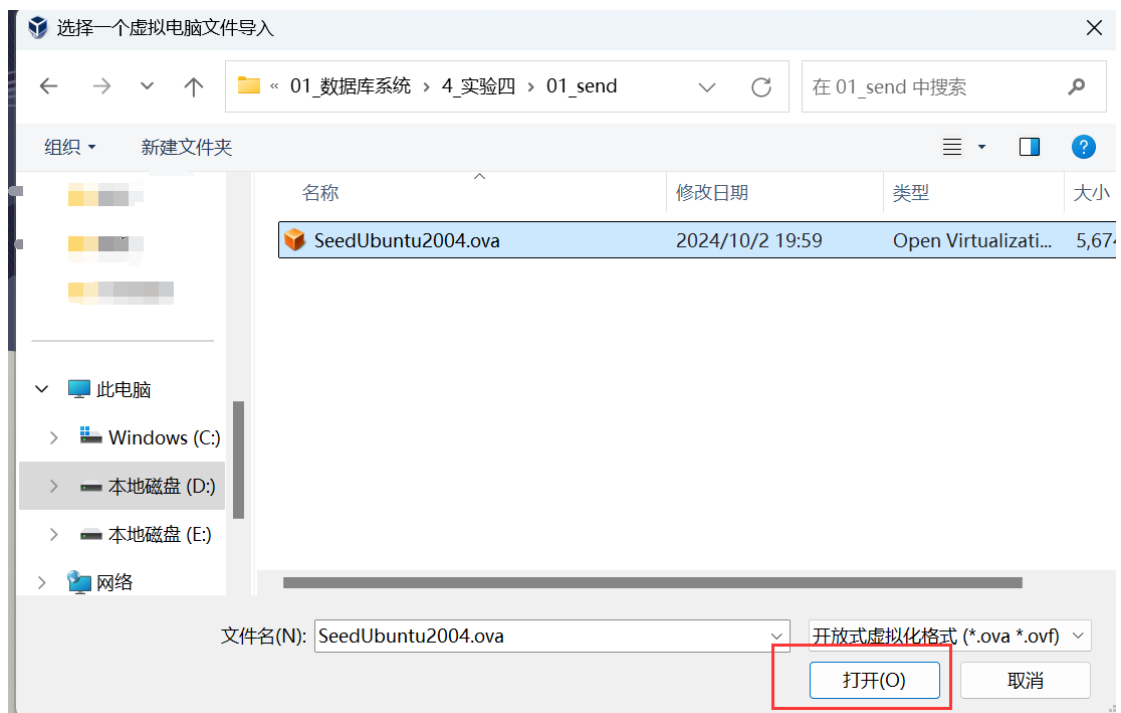


3、导入虚拟机





选择老师下发的虚拟机 ova 文件：





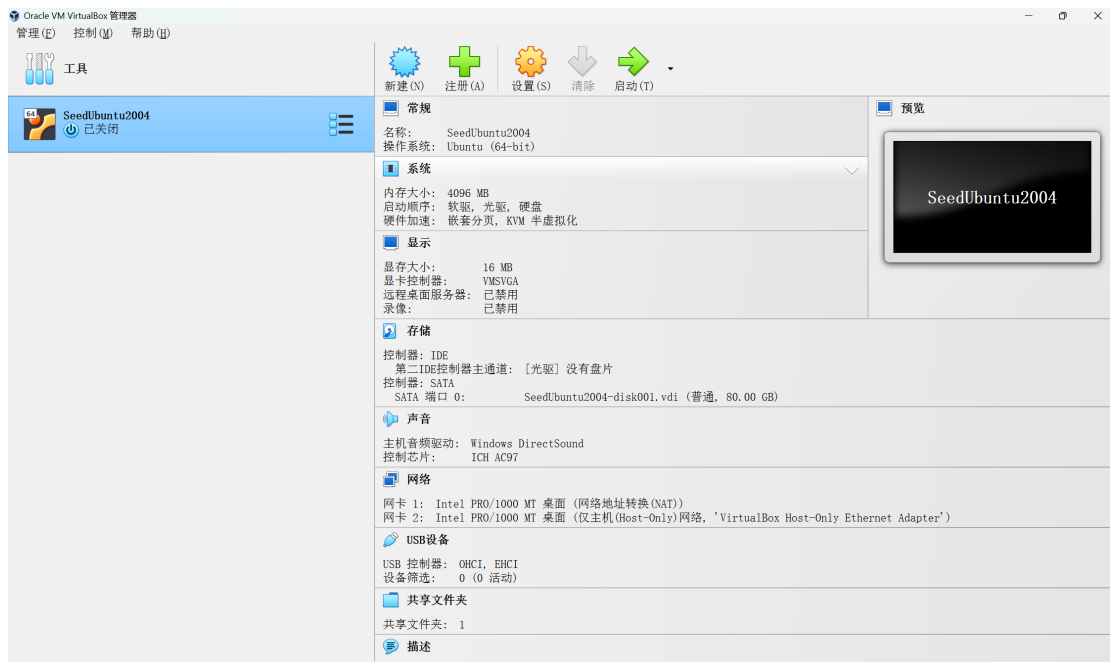
点击“下一步”：



点“完成”。



导入完毕：



4.1.2 使用虚拟机环境进行开发

1、启动、登录虚拟机

(1) 启动虚拟机



若启动出现如下报错：

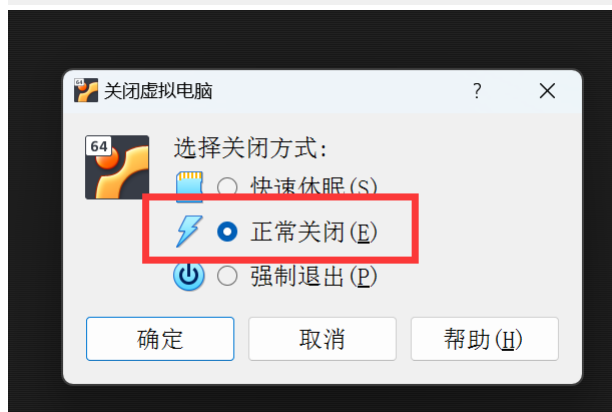
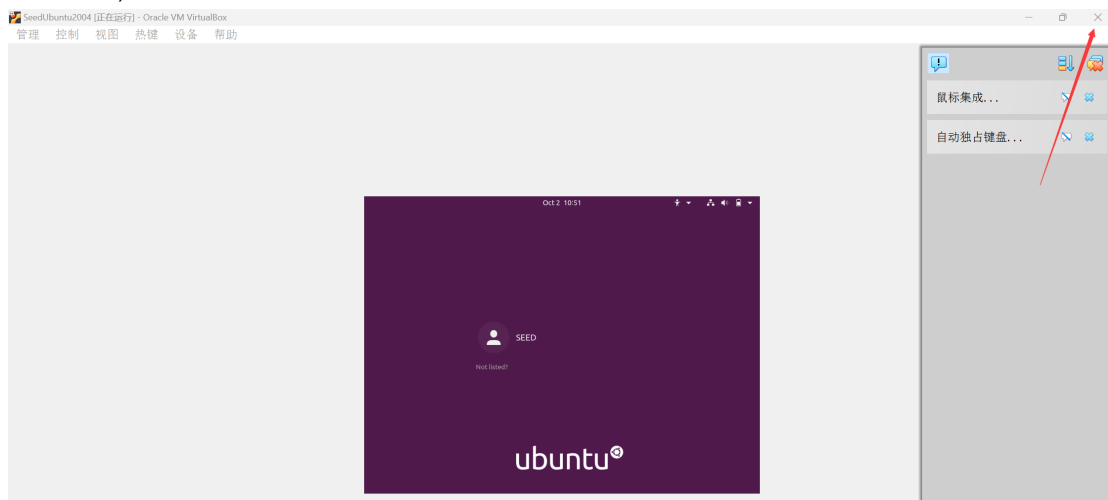


这是因为该虚拟机默认以：C:\Users\lenovo\Desktop\share 路径为共享路径，如果本机

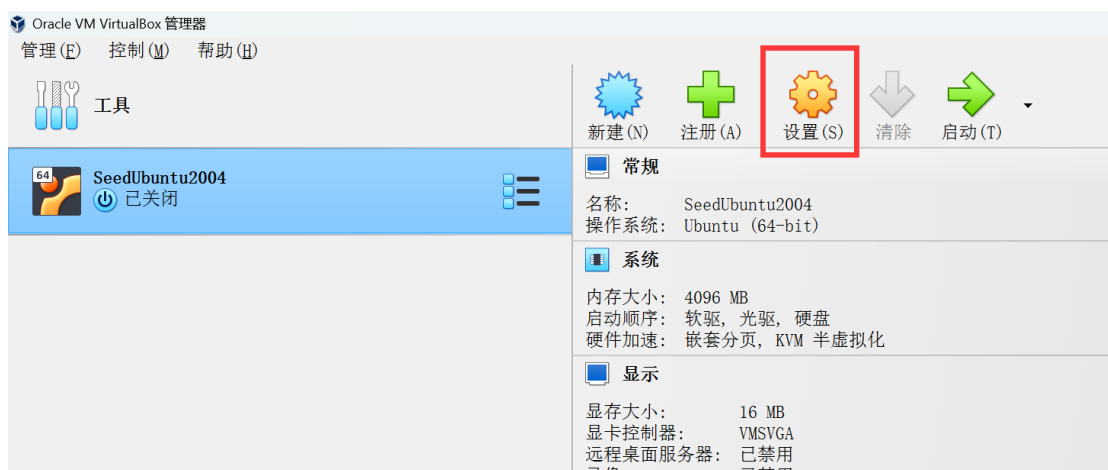
没有此路径需要重新选择共享路径。

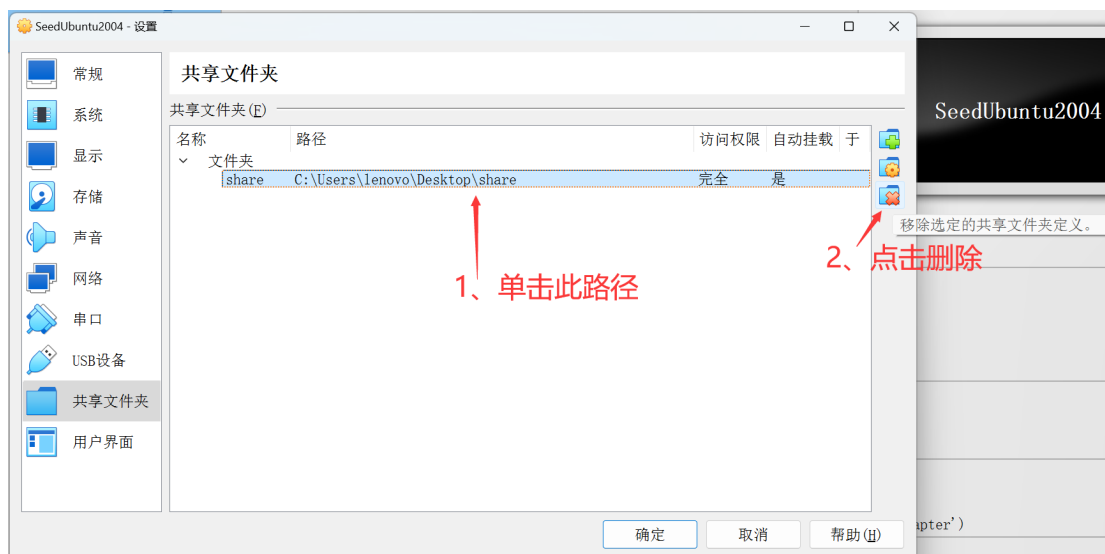
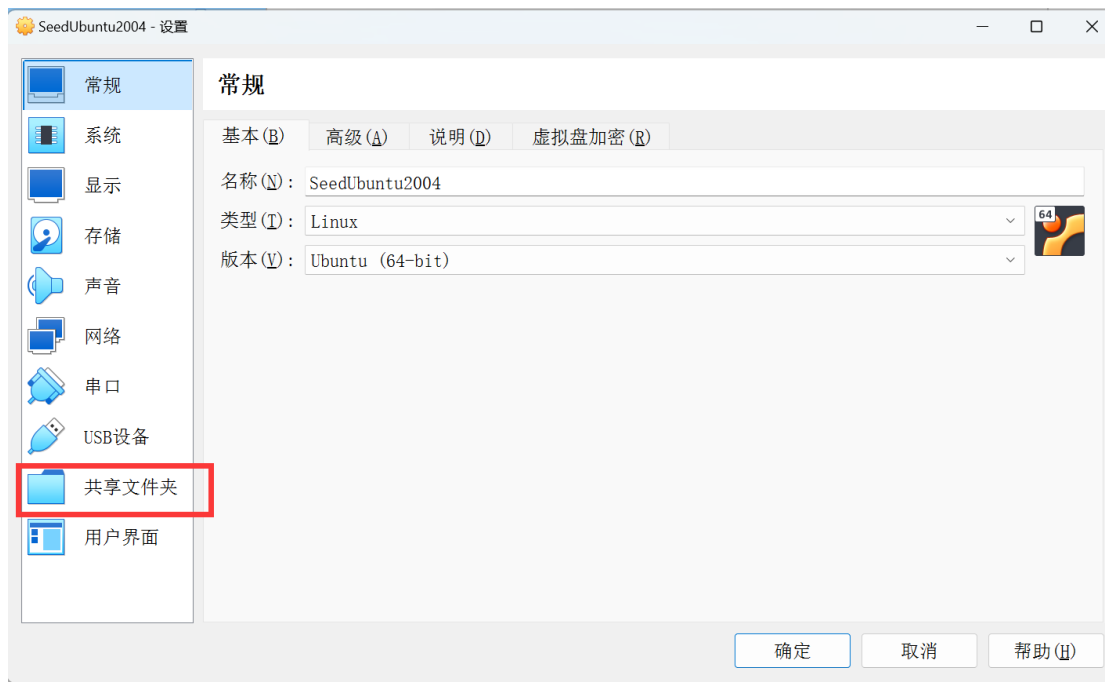
解决办法：

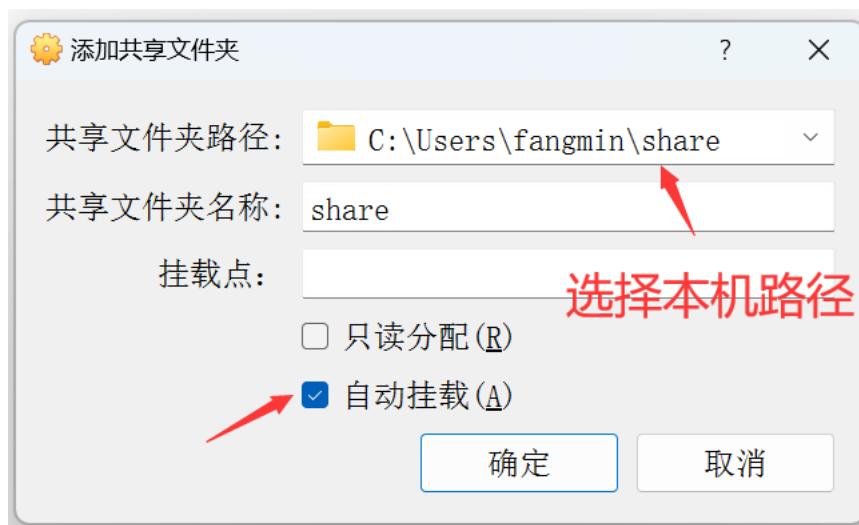
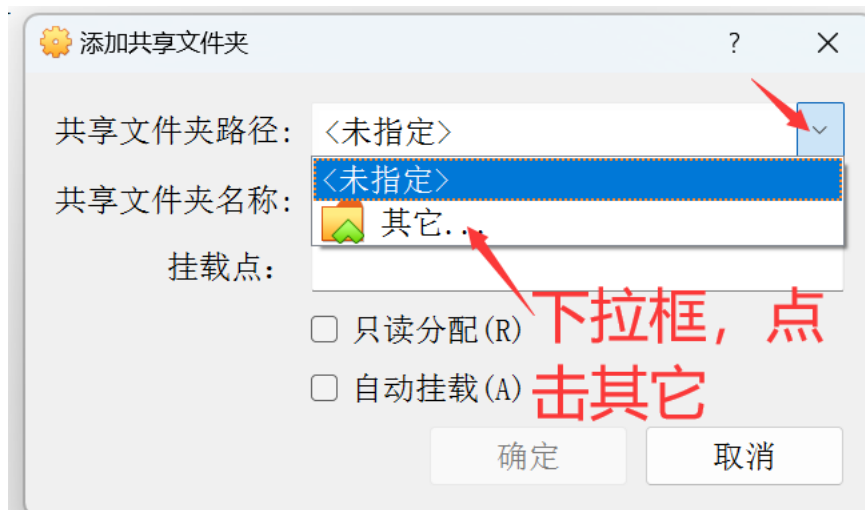
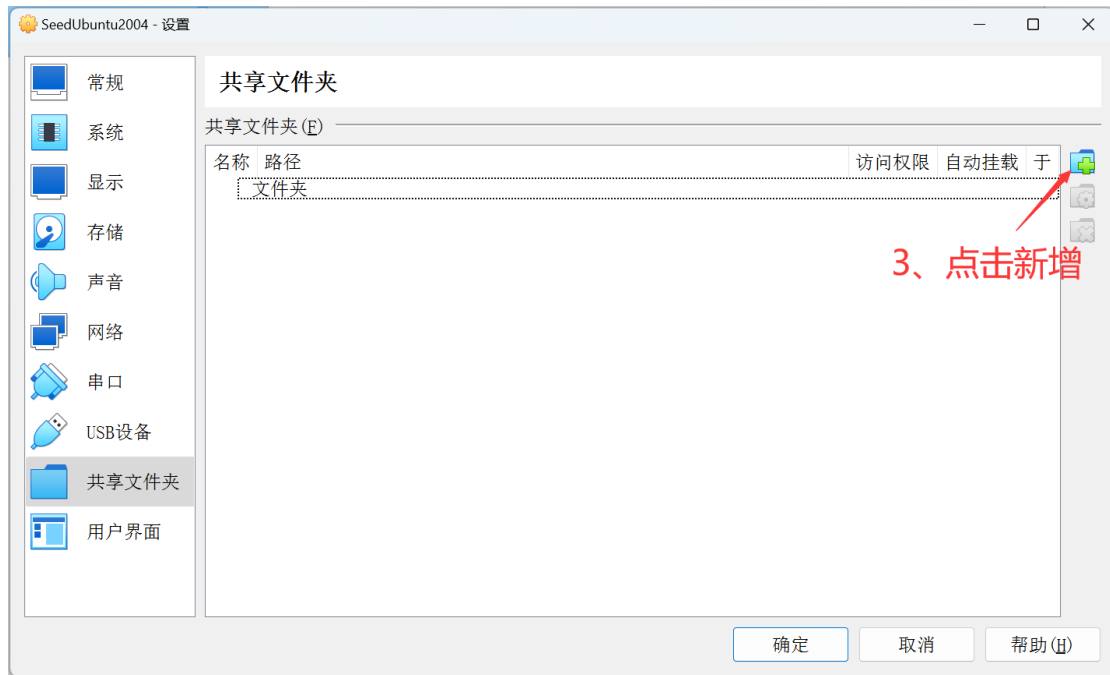
a) 关闭虚拟机



b) 在管理界面，点击“设置”：

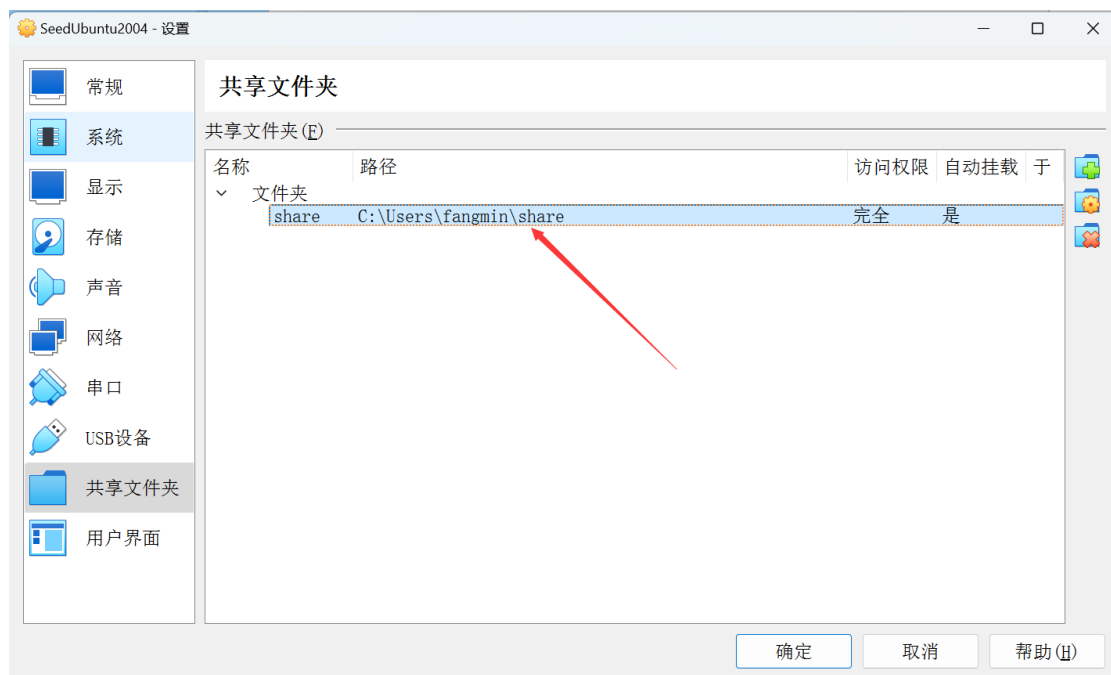




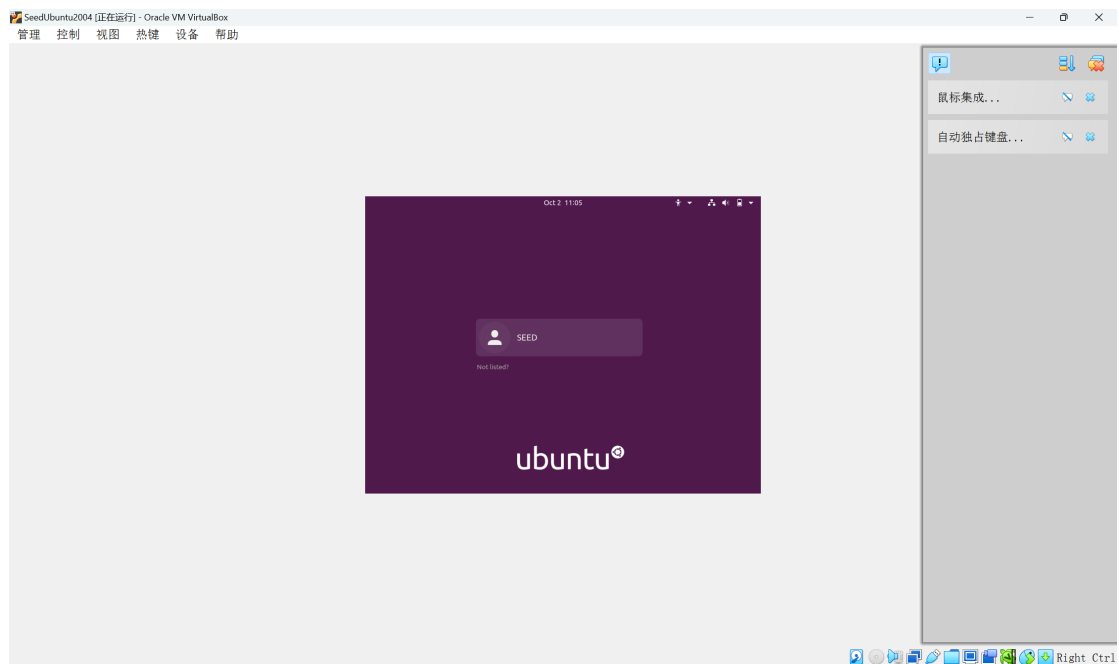


这个路径将成为本机和虚拟机交换数据的目录。

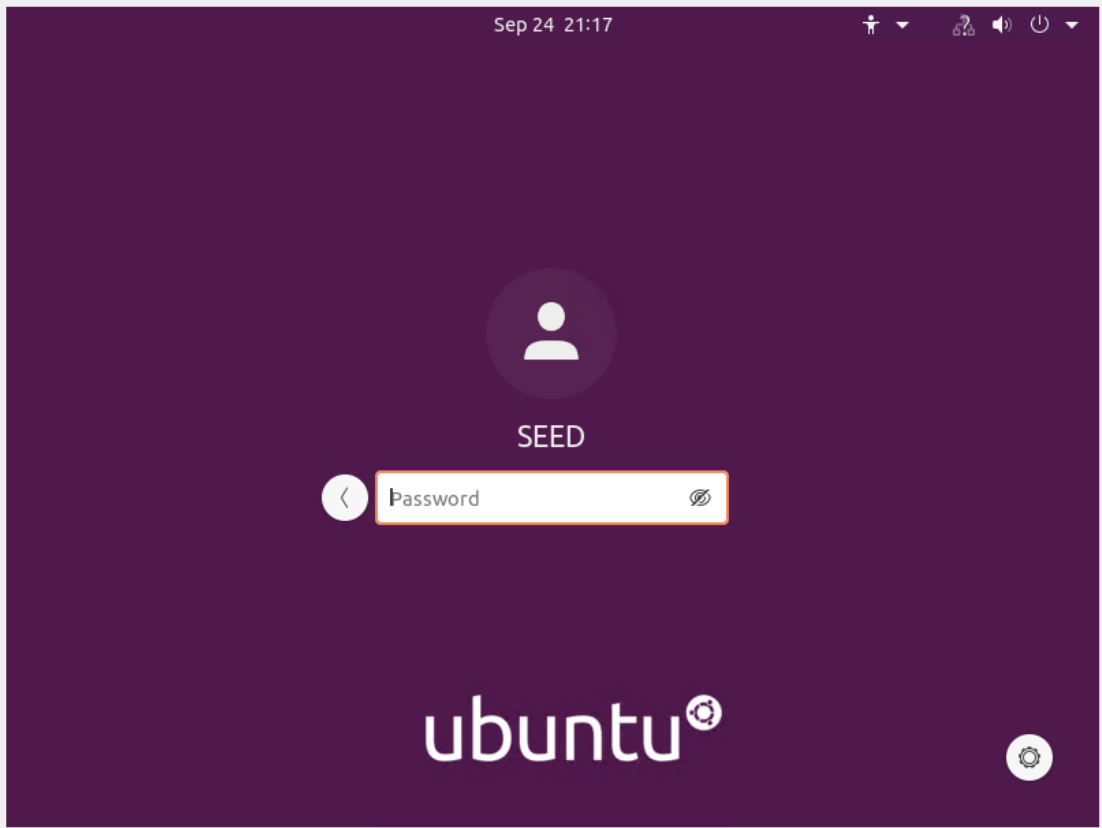
处理完毕：



c) 重新启动虚拟机：



(2) 登录



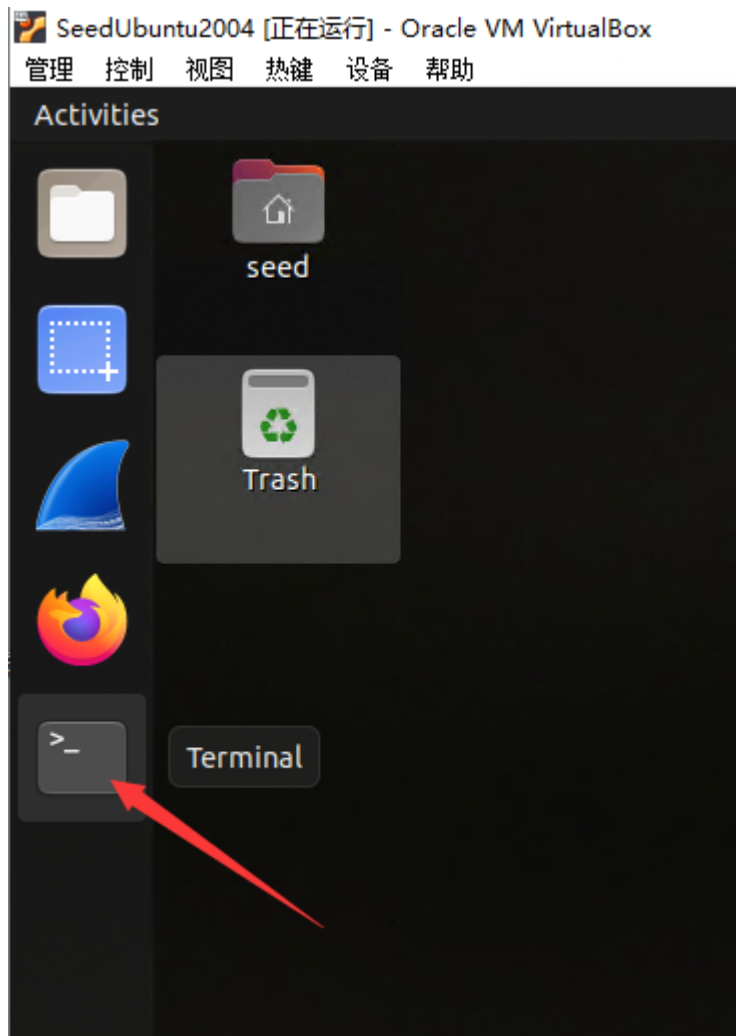
密码: dees

(3) 进行网络认证 (后续要从 github 上下载代码):



2、下载运行代码

(1) 打开终端:



(2) 下载框架代码

```
git clone --recursive https://github.com/ruc-deke/rucbase-lab.git
```

```
Terminal Sep 24 21:44
seed@VM: ~/.../20198001
[09/24/24]seed@VM:~$ cd temp 进入temp目录
[09/24/24]seed@VM:~/temp$ mkdir 20198001 新建属于你的文件夹
[09/24/24]seed@VM:~/temp$ cd 20198001 进入该文件夹并从github上下载框架代码
[09/24/24]seed@VM:~/.../20198001$ git clone --recursive https://github.com/ruc-deke/rucbase-lab.git
```

注意：有时网络不稳定无法从 github 上直接下载代码，可以使用老师提供的压缩包。方法请看附录 1。

(3) 查看下载的代码

```
[09/24/24]seed@VM:~/.../20198001$ ls
rucbase-lab
[09/24/24]seed@VM:~/.../20198001$ cd rucbase-lab
[09/24/24]seed@VM:~/.../rucbase-lab$ ls
CMakeLists.txt  deps  docs  LICENSE  pics  README.md  rucbase_client  src
```

(4) 进行 GoogleTest 子模块安装

a) 执行以下命令：

```
cd deps
cd googletest
mkdir build
cd build
cmake ..
```

```
[09/25/24]seed@VM:~/.../ruchbase-lab$ cd deps
[09/25/24]seed@VM:~/.../deps$ cd googletest
[09/25/24]seed@VM:~/.../googletest$ ls
BUILD.bazel  ci  CMakeLists.txt  CONTRIBUTING.md  CONTRIBUTORS  docs  googlemock  googletest  L
[09/25/24]seed@VM:~/.../googletest$ mkdir build
[09/25/24]seed@VM:~/.../googletest$ cd build
[09/25/24]seed@VM:~/.../build$ pwd
/home/seed/temp/20198001/ruchbase-lab/deps/googletest/build  注意要在此目录下执行
[09/25/24]seed@VM:~/.../build$ cmake ..
-- The C compiler identification is GNU 9.3.0
-- The CXX compiler identification is GNU 9.3.0
-- Check for working C compiler: /usr/bin/cc
-- Check for working C compiler: /usr/bin/cc -- works
-- Detecting C compiler ABI info
-- Detecting C compiler ABI info - done
-- Detecting C compile features
-- Detecting C compile features - done
```

b) 执行 make 命令:

```
[09/25/24]seed@VM:~/.../build$ make
Scanning dependencies of target gtest
[ 12%] Building CXX object googletest/CMakeFiles/gtest.dir/src/gtest-all.cc.o
[ 25%] Linking CXX static library ../lib/libgtest.a
[ 25%] Built target gtest
Scanning dependencies of target gmock
[ 37%] Building CXX object googletest/CMakeFiles/gmock.dir/src/gmock-all.cc.o
[ 50%] Linking CXX static library ../lib/libgmock.a
[ 50%] Built target gmock
Scanning dependencies of target gmock_main
[ 62%] Building CXX object googletest/CMakeFiles/gmock_main.dir/src/gmock_main.cc.o
[ 75%] Linking CXX static library ../lib/libgmock_main.a
[ 75%] Built target gmock_main
Scanning dependencies of target gtest_main
[ 87%] Building CXX object googletest/CMakeFiles/gtest_main.dir/src/gtest_main.cc.o
[100%] Linking CXX static library ../lib/libgtest_main.a
[100%] Built target gtest_main
```

c) 执行 sudo make install 命令:

```
[09/25/24]seed@VM:~/.../build$ sudo make install
[ 25%] Built target gtest
[ 50%] Built target gmock
[ 75%] Built target gmock_main
[100%] Built target gtest_main
Install the project...
-- Install configuration: ""
-- Up-to-date: /usr/local/include
-- Installing: /usr/local/include/gmock
-- Installing: /usr/local/include/gmock/gmock-matchers.h
-- Installing: /usr/local/include/gmock/gmock-function-mocker.h
-- Installing: /usr/local/include/gmock/gmock-more-matchers.h
-- Installing: /usr/local/include/gmock/gmock-nice-strict.h
-- Installing: /usr/local/include/gmock/gmock-actions.h
-- Installing: /usr/local/include/gmock/internal
-- Installing: /usr/local/include/gmock/internal/custom
```

(5) 代码分服务端和客户端，先编译服务端：

- a) 在 rucbase-lab 目录下新建 build 目录
- b) 进入 build 目录执行 cmake ..

```
[09/24/24] seed@VM:~/.../rucbase-lab$ mkdir build
[09/24/24] seed@VM:~/.../rucbase-lab$ cd build
[09/24/24] seed@VM:~/.../build$ cmake ..
-- The C compiler identification is GNU 9.3.0
-- The CXX compiler identification is GNU 9.3.0
-- Check for working C compiler: /usr/bin/cc
-- Check for working C compiler: /usr/bin/cc -- works
-- Detecting C compiler ABI info
-- Detecting C compiler ABI info - done
-- Detecting C compile features
-- Detecting C compile features - done
-- Check for working CXX compiler: /usr/bin/c++
-- Check for working CXX compiler: /usr/bin/c++ -- works
-- Detecting CXX compiler ABI info
-- Detecting CXX compiler ABI info - done
-- Detecting CXX compile features
-- Detecting CXX compile features - done
```

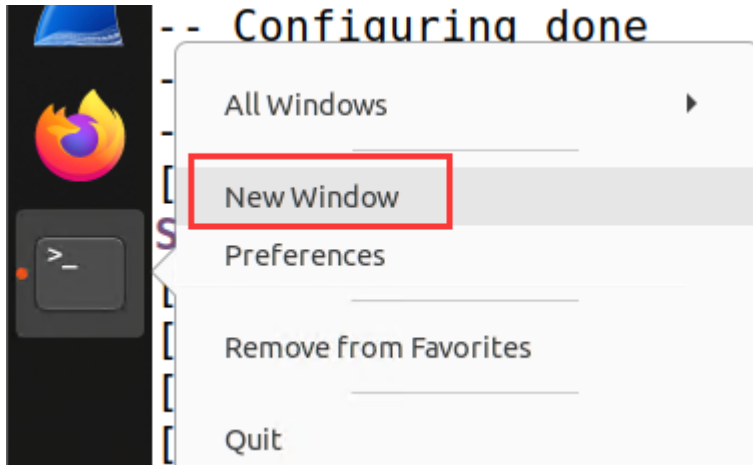
- c) 执行 make rmdb -j4

```
[09/24/24] seed@VM:~/.../build$ make rmdb -j4
Scanning dependencies of target storage
Scanning dependencies of target analyze
Scanning dependencies of target planner
[ 3%] Building CXX object src/storage/CMakeFiles/storage.dir/disk_manager.cpp.o
[ 9%] Building CXX object src/analyze/CMakeFiles/analyze.dir/analyze.cpp.o
[ 6%] Building CXX object src/storage/CMakeFiles/storage.dir/buffer_pool_manager.cpp.o
[ 12%] Building CXX object src/optimizer/CMakeFiles/planner.dir/planner.cpp.o
/home/seed/temp/20198001/rucbase-lab/src/storage/disk_manager.cpp: In member function 'int DiskManager::open_file(
/home/seed/temp/20198001/rucbase-lab/src/storage/disk_manager.cpp:129:1: warning: no return statement in function
n-type]
129 | }
    | ^
[ 15%] Building CXX object src/storage/CMakeFiles/storage.dir/_replacer/lru_replacer.cpp.o
In file included from /home/seed/temp/20198001/rucbase-lab/src/index/ix_manager.h:16,
                 from /home/seed/temp/20198001/rucbase-lab/src/index/ix.h:14,
                 from /home/seed/temp/20198001/rucbase-lab/src/system/sm_manager.h:13,
                 from /home/seed/temp/20198001/rucbase-lab/src/system/sm.h:13,
                 from /home/seed/temp/20198001/rucbase-lab/src/execution/execution_manager.h:21,
                 from /home/seed/temp/20198001/rucbase-lab/src/optimizer/planner.h:20,
                 from /home/seed/temp/20198001/rucbase-lab/src/optimizer/planner.cpp:11:
/home/seed/temp/20198001/rucbase-lab/src/system/sm_meta.h: In member function 'bool TabMeta::is_index(const std::v
tring&name, sf) const':
~~~~~
[100%] Linking CXX executable ../bin/rmdb
[100%] Built target rmdb
```

← 生成的rmdb为服务端

(6) 下面编译客户端：

- a) 进入 rucbase_client 目录，新建 build 文件夹
- b) 进入 build 文件夹，执行 cmake ..



```
[09/24/24] seed@VM:~/.../rucbase-lab$ cd rucbase_client/build
[09/24/24] seed@VM:~/.../build$ pwd
/home/seed/temp/20198001/rucbase-lab/rucbase_client/build
[09/24/24] seed@VM:~/.../build$ ./rucbase_client
Rucbase>
```

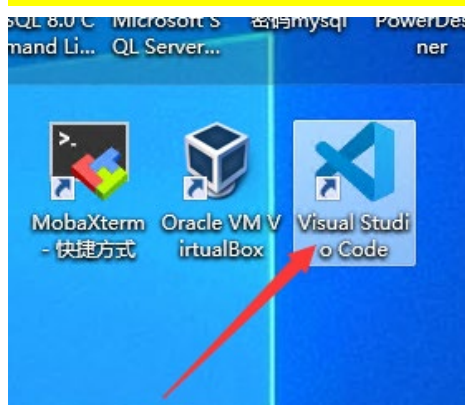
到rucbase_client/build目录下

执行 show tables; 命令:

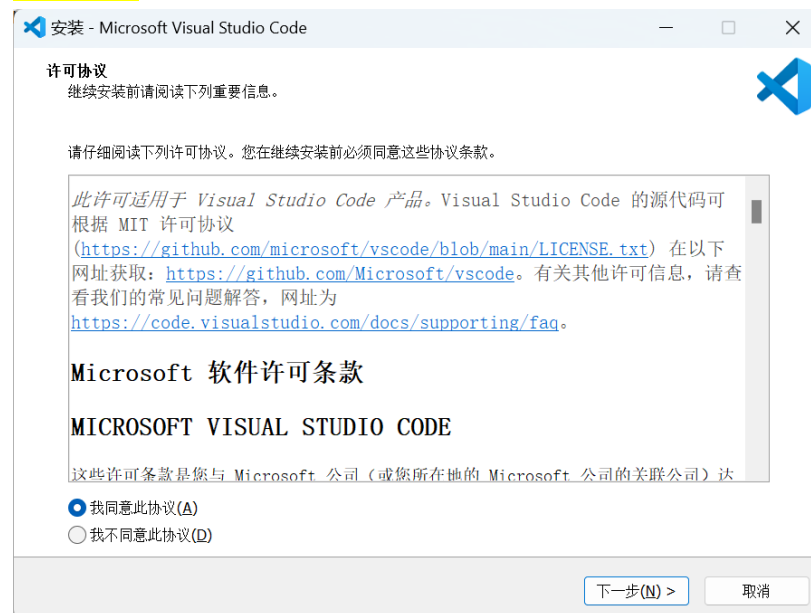
```
[09/24/24] seed@VM:~/.../build$ ./rucbase_client
Rucbase> show tables;
+-----+
| Tables |
+-----+
Rucbase>
```

3、安装 VSCode

如果使用实验室环境，可跳过本节。要注意实验室电脑上安装了 2 个版本的 VSCode，要用在 Oracle VM VirtualBox 旁边的 VSCode:



如果你本机已安装 VSCode，你可以尝试下面“使用 VSCode 打开远程代码”的操作，如果连不上虚拟机，可考虑使用老师下发的 VSCode 重新安装。（有时连不上是因为 VSCode 的版本问题）。



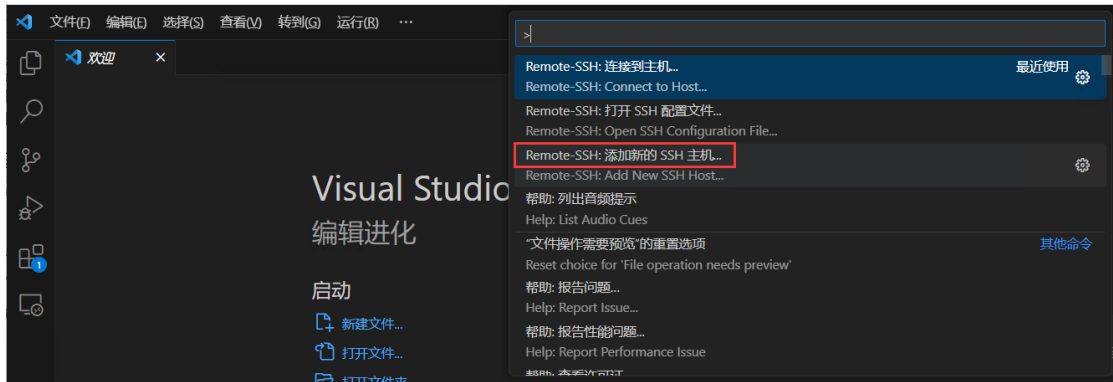
4、使用 VSCode 打开远程代码

- (1) 我们要知道主机的 ip，在虚拟机的终端执行 ip add:

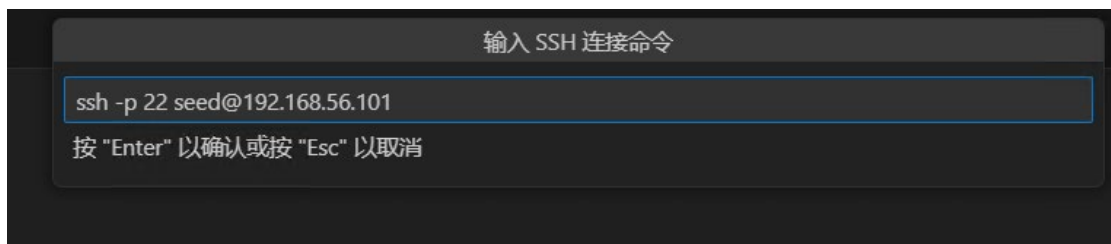
```
Activities Terminal Sep 24 23:26
seed@VM: ~
[09/24/24]seed@VM:~$ ip add
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:d8:91:40 brd ff:ff:ff:ff:ff:ff
    inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic noprefixroute enp0s3
        valid_lft 85951sec preferred_lft 85951sec
    inet6 fe80::8289:3aa:79e5:9d10/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
3: enp0s8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:65:14:5a brd ff:ff:ff:ff:ff:ff
    inet 192.168.56.101/24 brd 192.168.56.255 scope global dynamic noprefixroute enp0s8
        valid_lft 451sec preferred_lft 451sec
    inet6 fe80::6759:2f0:2cab:e849/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
4: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default
    link/ether 02:42:93:9e:9a:6d brd ff:ff:ff:ff:ff:ff
    inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
        valid_lft forever preferred_lft forever
[09/24/24]seed@VM:~$
```

这里显示的 192.168.56.101 就是虚拟机的 ip。

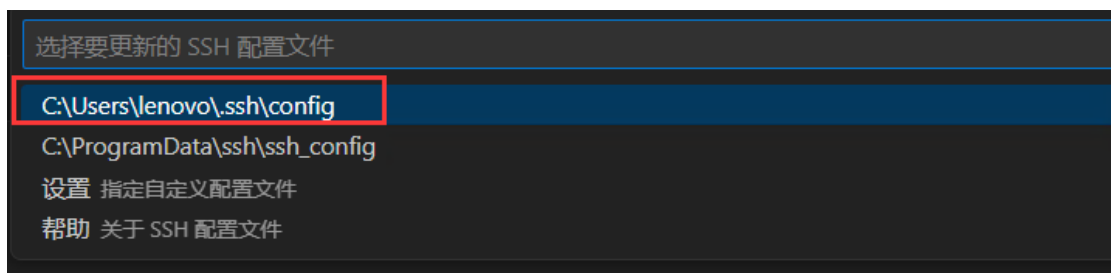
- (2) 回到本地，打开 VScode，按下 F1（或者在界面左下角找到齿轮→命令面板），输入“Remote-SSH”，选择 Remote-SSH:添加新的 SSH 主机：



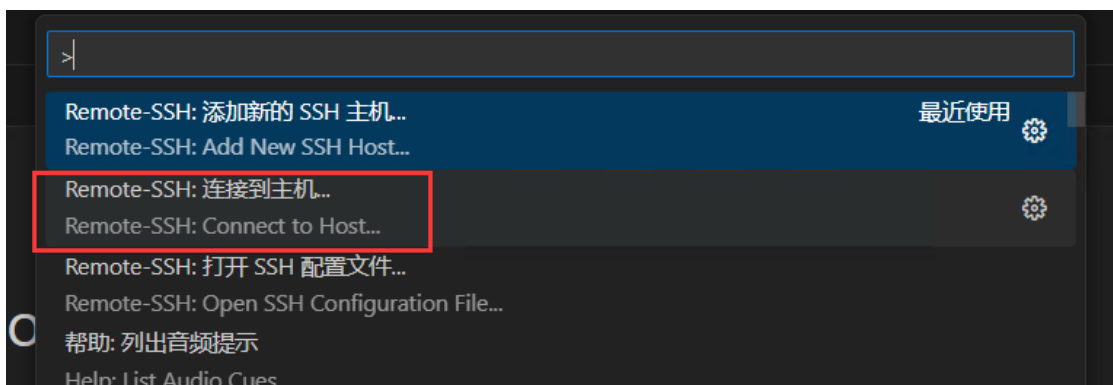
- (3) 输入 `ssh -p 22 seed@192.168.56.101`, 注意这里 ip 要替换成你的 ip:



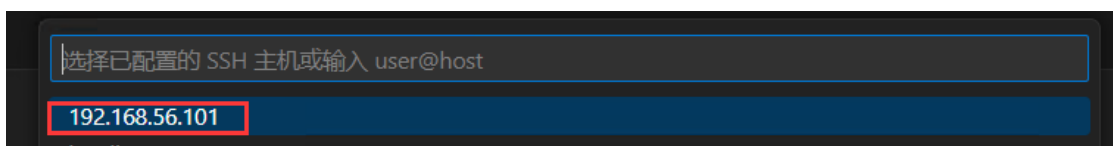
将配置保存在配置文件中:

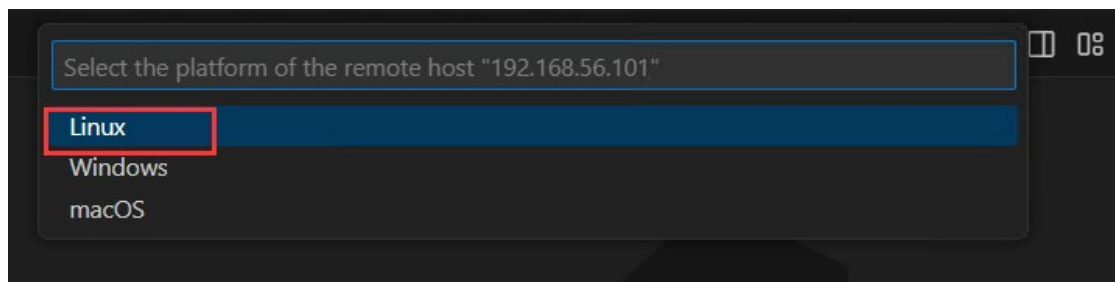
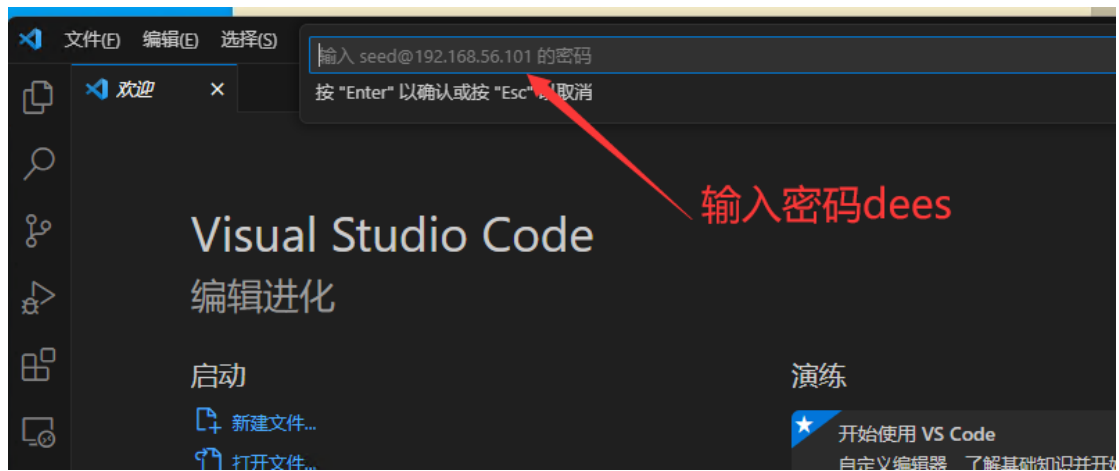


- (4) 按下 F1 键, 选择 Remote-SSH:连接到主机:

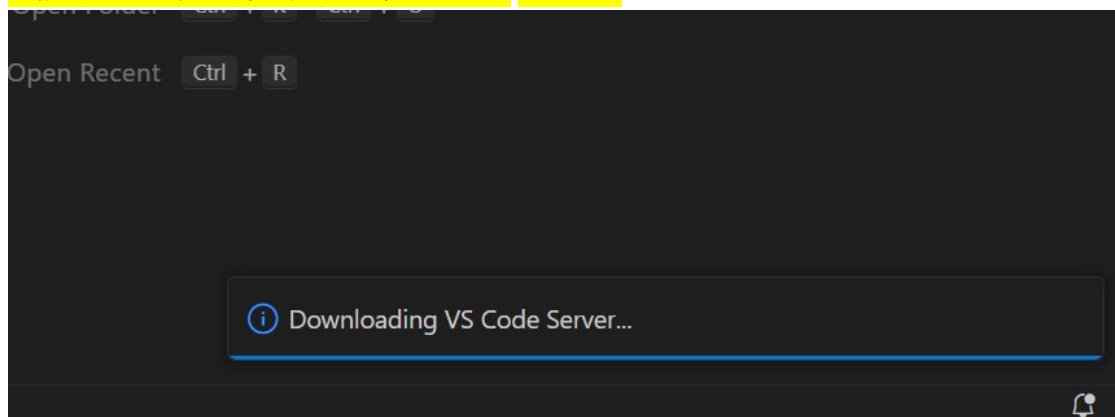


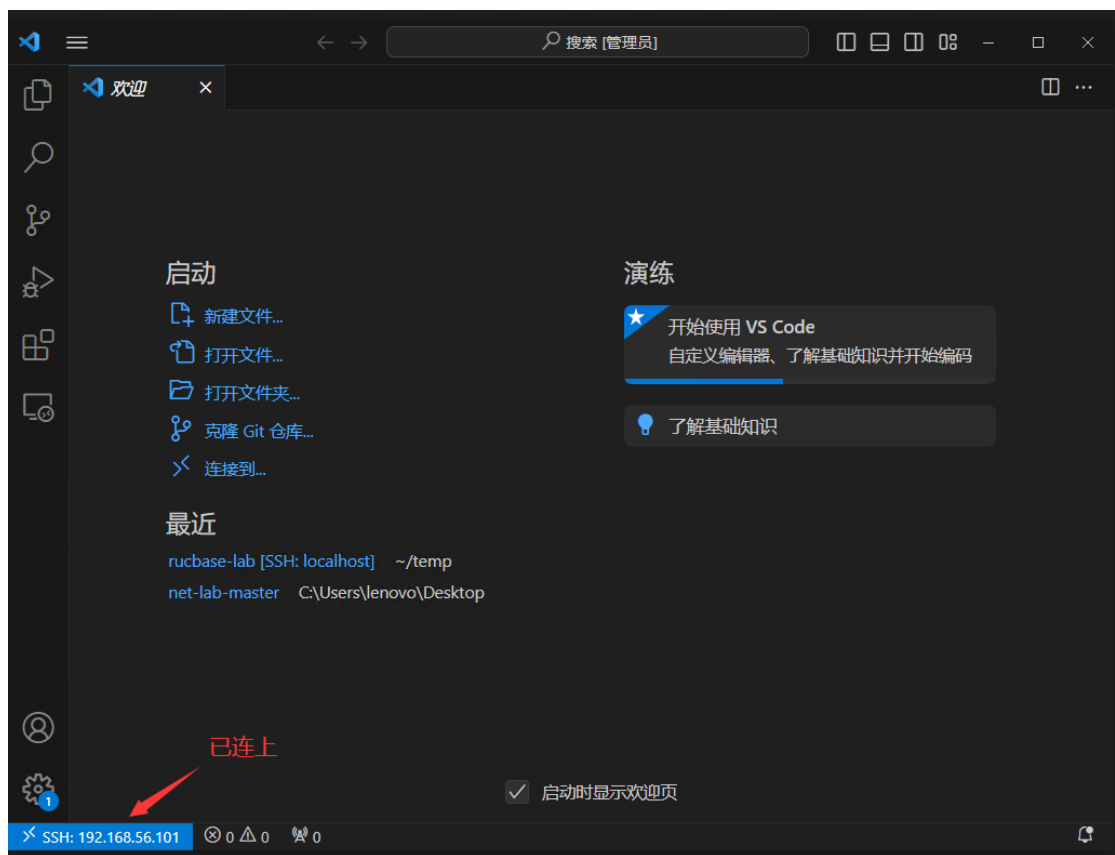
选择刚配置的 192.168.56.101 :



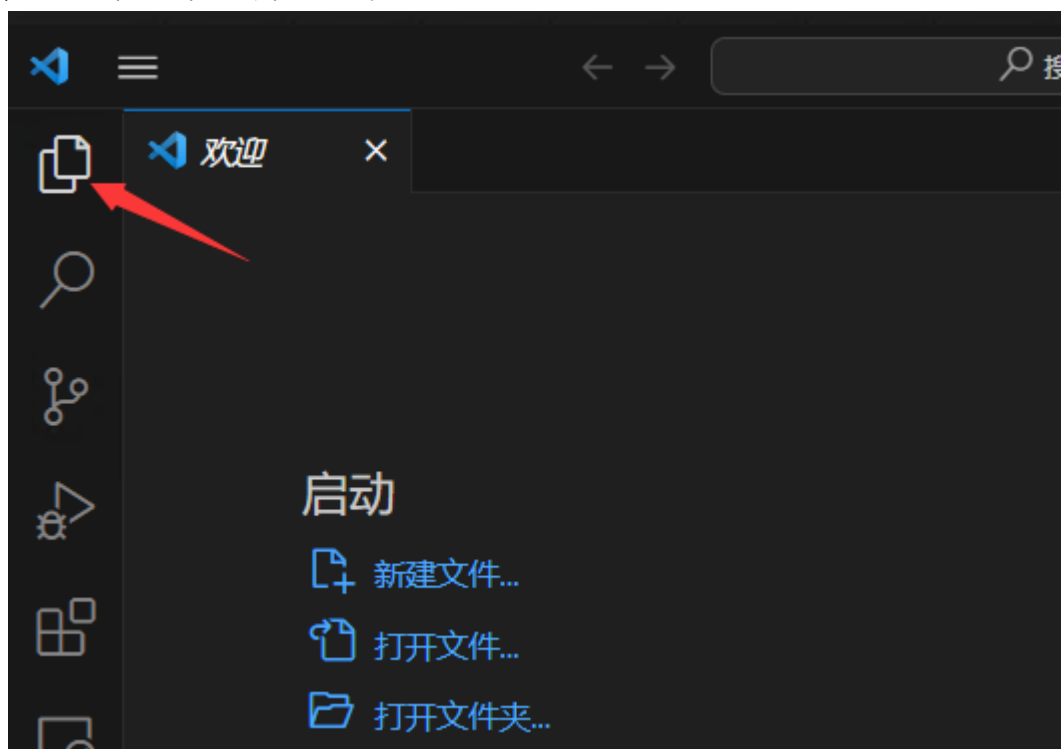


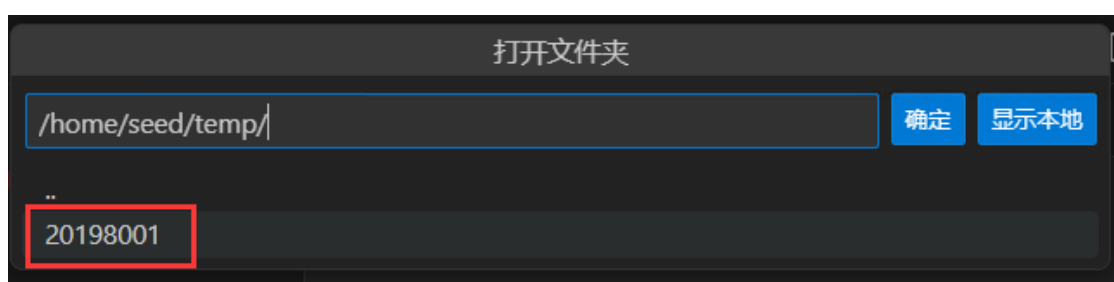
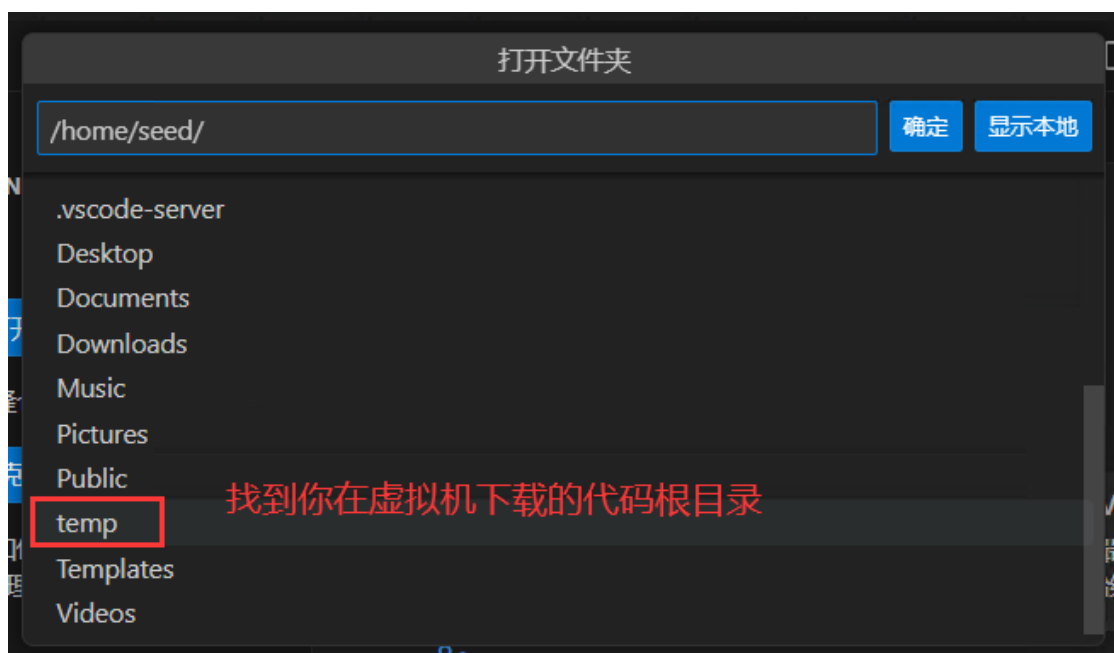
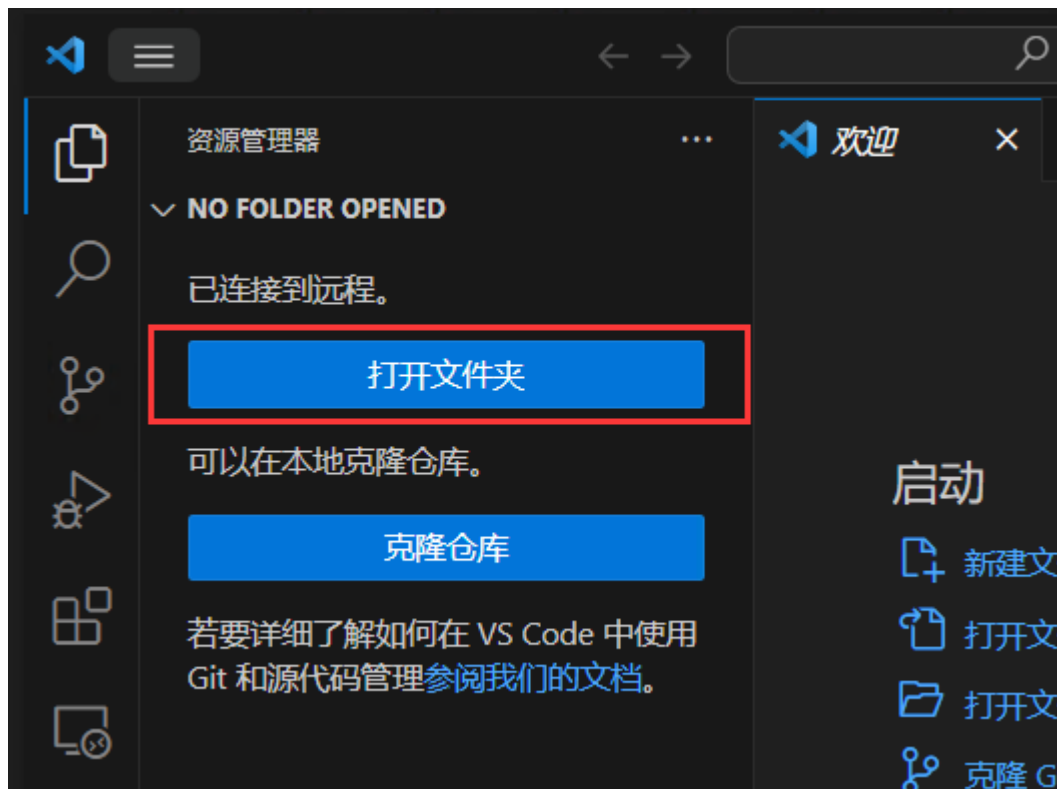
连接会比较慢，因为要安装一个 VS Code Server:

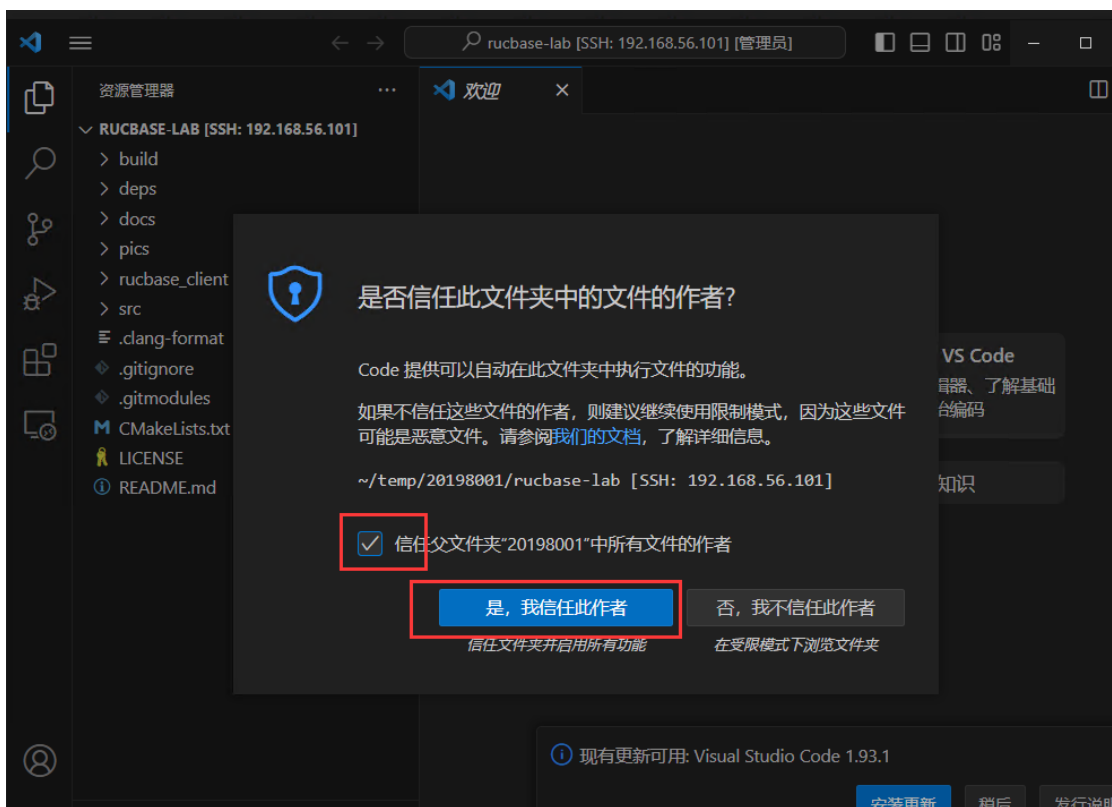
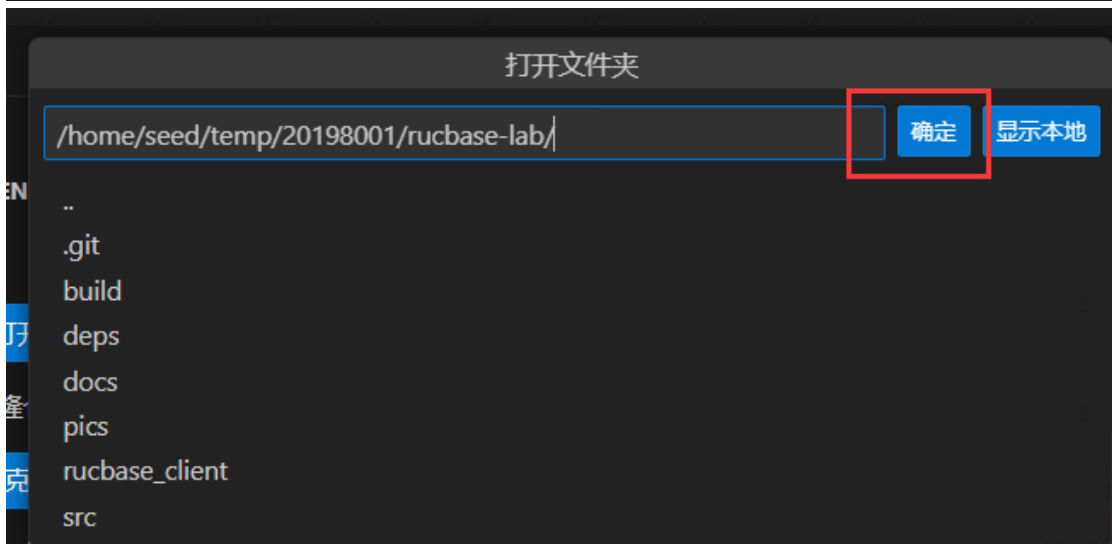
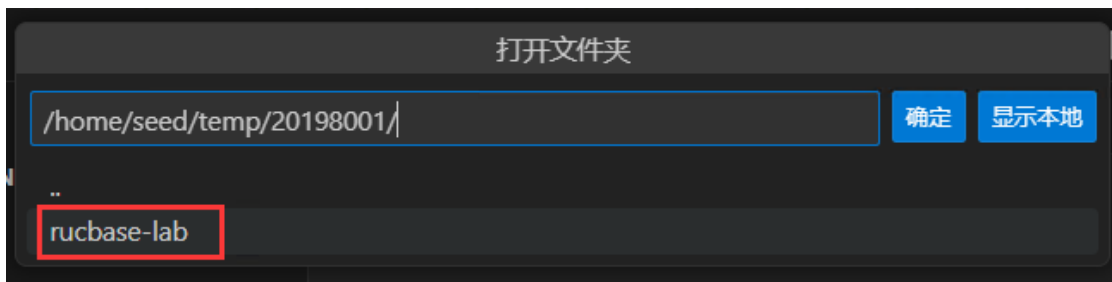




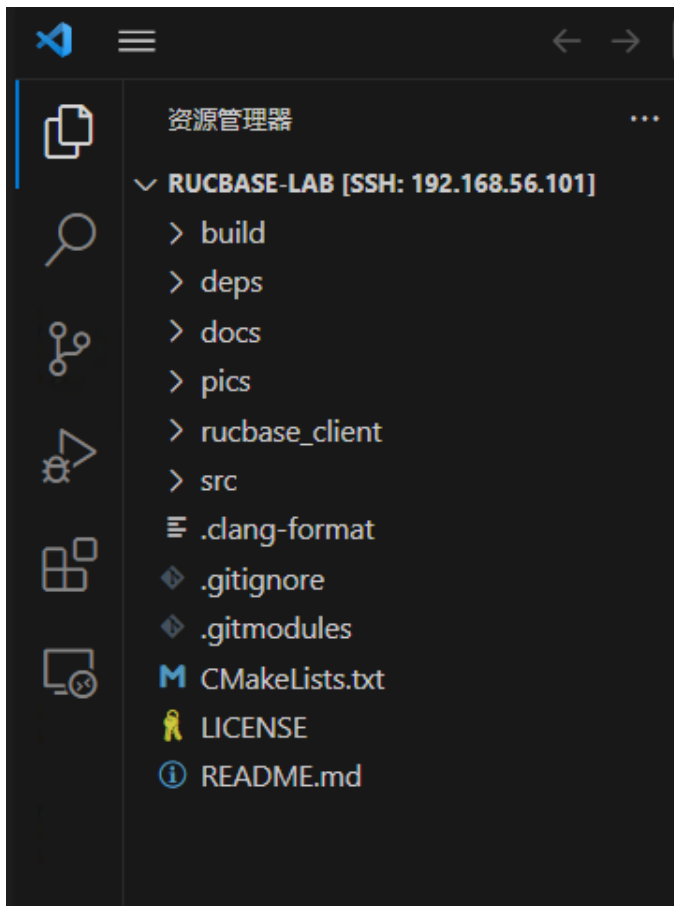
(5) 选择打开在虚拟机下载的 rucbase-lab 目录：







可以看到代码已经显示在 VSCode 中：

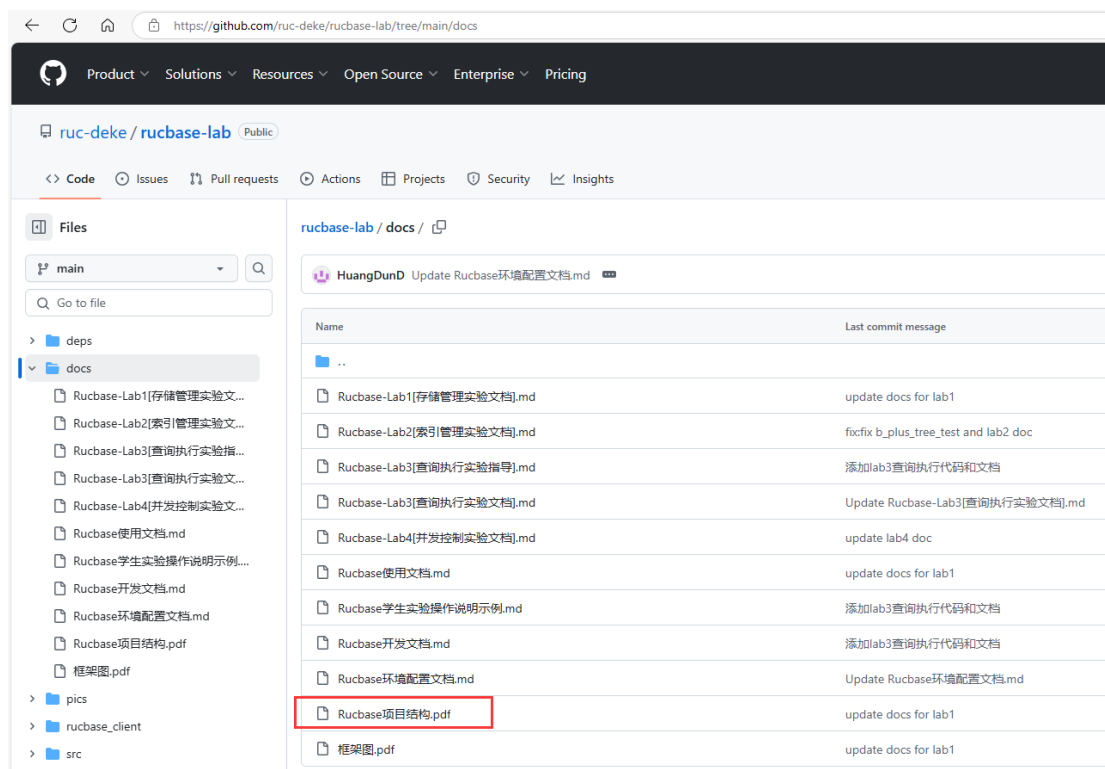


下面就可以开始做开发工作了。

4.2 开发

我们在进行开发前，可以通过相关文档了解框架代码及实验任务。

1. 了解框架代码: <https://github.com/ruc-deke/rucbase-lab/tree/main/docs>



打开文档，我们可以优先看与本次实验任务相关的部分：

RMDB项目结构详解

RMDB项目结构详解

代码框架

存储管理(Storage Management)

相关知识点

项目结构

文件存储组织模块：src/storage

Page

DiskManager

BufferPoolManager

缓冲区管理：src/replacer

记录存储组织模块：src/record

元数据存储组织：src/system

索引(Index)

相关知识点

项目结构

并发控制(Concurrency control)

相关知识点

项目结构

故障恢复(Failure recovery)

相关知识点

项目结构

查询处理与执行(Query processing and execution)

相关知识点

项目结构

语法解析

语法树结构

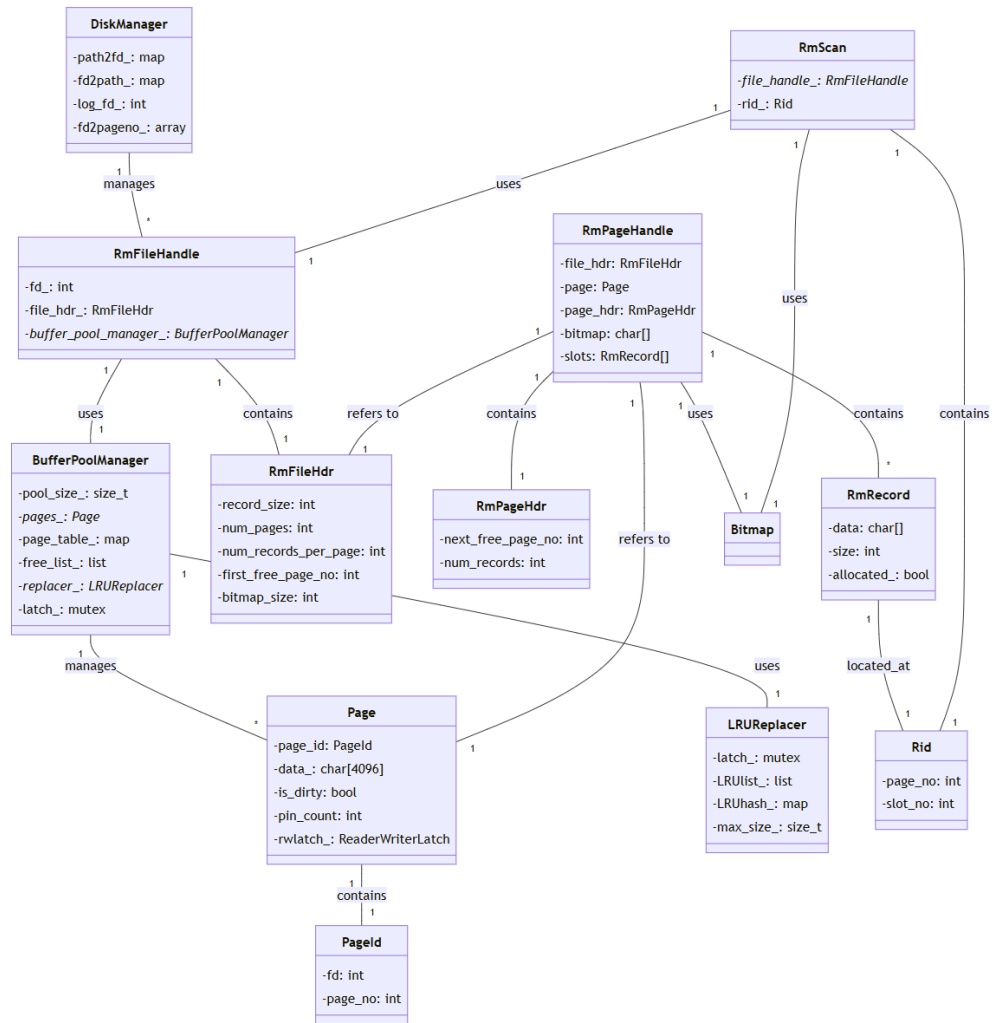
错误与异常处理

2. 查看本次实验任务：[Rucbase-Lab1\[存储管理实验文档\]](#)

[rucbase-lab](#) / docs /

 HuangDunD Update Rucbase环境配置文档.md	
Name	
 ..	
 Rucbase-Lab1[存储管理实验文档].md	
 Rucbase-Lab2[索引管理实验文档].md	
 Rucbase-Lab3[查询执行实验指导].md	
 Rucbase-Lab3[查询执行实验文档].md	
 Rucbase-Lab4[并发控制实验文档].md	
 Rucbase使用文档.md	
 Rucbase学生实验操作说明示例.md	
 Rucbase开发文档.md	
 Rucbase环境配置文档.md	
 Rucbase项目结构.pdf	
 框架图.pdf	

理清框架代码中多个数据结构以及它们之间的关系：



这里，DiskManager、LRURemplacer、BufferPoolManager、RMFileHandle 和 RmScan 是有 ToDo 任务的、需要修改的类。其它类需了解。

3. 打开 [Rucbase-Lab1\[存储管理实验文档\]](#):

- [存储管理实验文档](#)
 - [任务一 缓冲池管理器](#)
 - [任务1.1 磁盘存储管理器](#)
 - [任务1.2 缓冲池替换策略](#)
 - [任务1.3 缓冲池管理器](#)
 - [任务二 记录管理器](#)
 - [任务2.1 记录操作](#)
 - [任务2.2 记录迭代器](#)
 - [实验计分](#)

下面我们以任务 1.1 为例示范开发和测试。

4. 在 [Rucbase-Lab1\[存储管理实验文档\]](#)中查看任务 1.1:

任务1.1 磁盘存储管理器

本任务要求补全 `DiskManager` 类，其负责读写指定页面、分配页面编号，以及对文件进行操作。

`DiskManager` 类的接口如下：

```
class DiskManager {
public:
    explicit DiskManager();           // Constructor
    ~DiskManager() = default;         // Destructor
    // 读写页面
    void write_page(int fd, page_id_t page_no, const char *offset, int num_bytes);
    void read_page(int fd, page_id_t page_no, char *offset, int num_bytes);
    // 对指定文件分配页面编号
    page_id_t allocate_page(int fd);
    // 文件操作
    bool is_file(const std::string &path);
    void create_file(const std::string &path);
    int open_file(const std::string &path);
    void close_file(int fd);
    void destroy_file(const std::string &path);
}
```

这些接口的内部实现调用了Linux操作系统下 `/usr/include/unistd.h` 提供的 `read()`、`write()`、`open()`、`close()`、`unlink()` 等函数。

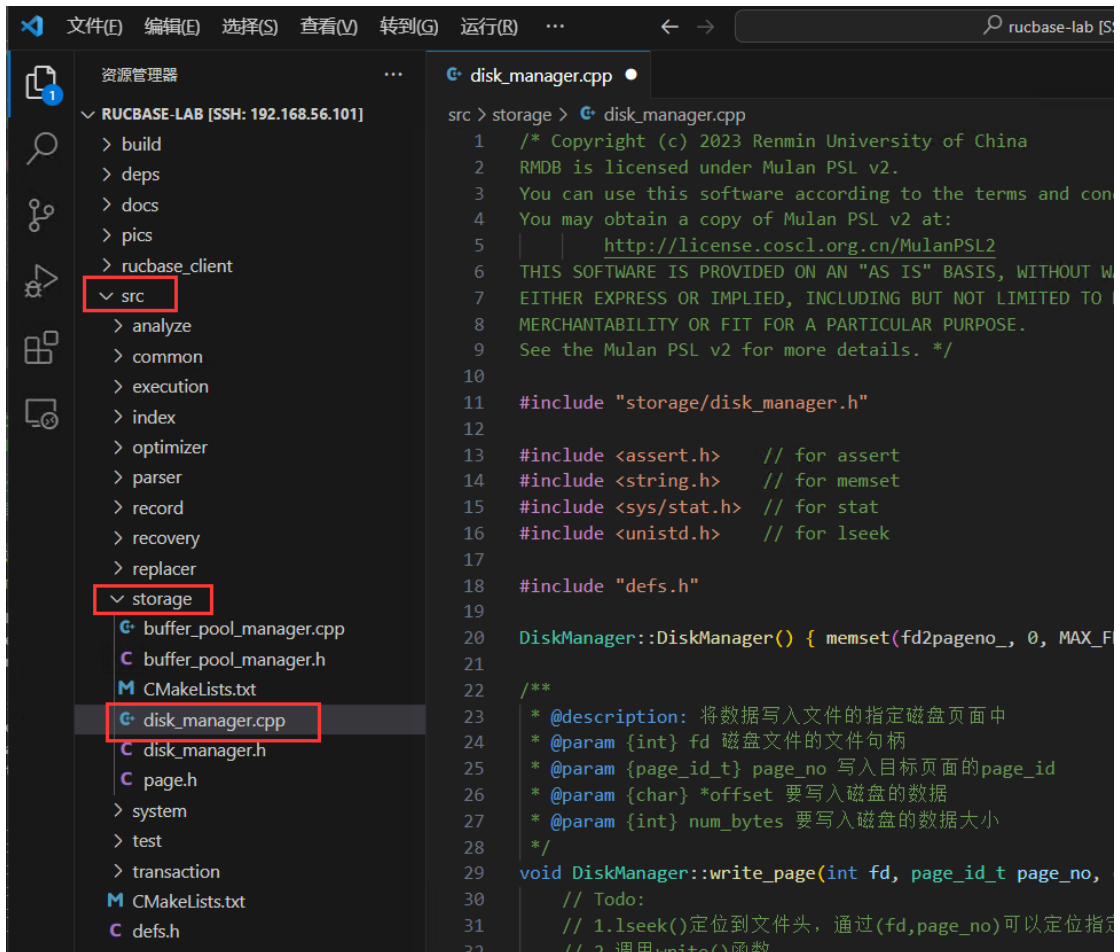
(1) 读写页面

- `void write_page(int fd, page_id_t page_no, const char *offset, int num_bytes);`
- `void read_page(int fd, page_id_t page_no, char *offset, int num_bytes);`

提示：可以调用 `read()` 或 `write()` 函数。通过 `(fd, page_no)` 可以定位指定页面及其在磁盘文件中的偏移量。注意：这里支持读写的字节长度为 `num_bytes`，上层调用此函数读写页面时，其值一般为页面大小 `PAGE_SIZE`。但有时也可以小于 `PAGE_SIZE`，比如只读写页头数据。

(2) 分配页面编号

5. 在框架代码中找到 `DiskManager` 类：



按要求补全，注意保存代码。

4.3 测试

1. 下面要进行测试，在 [Rucbase-Lab1\[存储管理实验文档\]](#) 文档最后有关于测试的部分：

rucbase-lab / docs / Rucbase-Lab1[存储管理实验文档].md

Preview Code Blame 415 lines (253 loc) · 17.1 KB

可以自定义末尾的标识符，如 RM_NO_PAGE。

(4) Rid rid() const override;

获取 RmScan 内部存放的 rid。

实验计分

在本实验中，每个任务对应一个单元测试文件，每个测试文件中包含若干测试点。通过测试点即可得分，满分为100分。

测试文件及测试点如下：

任务点	测试文件	分值
任务1.1 磁盘存储管理器	src/test/storage/disk_manager_test.cpp	10
任务1.2 缓冲池替换策略	src/test/storage/lru_replacer_test.cpp	20
任务1.3 缓冲池管理器	src/test/storage/buffer_pool_manager_test.cpp	40
任务2 记录管理器	src/test/storage/record_manager_test.cpp	30

编译生成可执行文件进行测试：

```
cd build

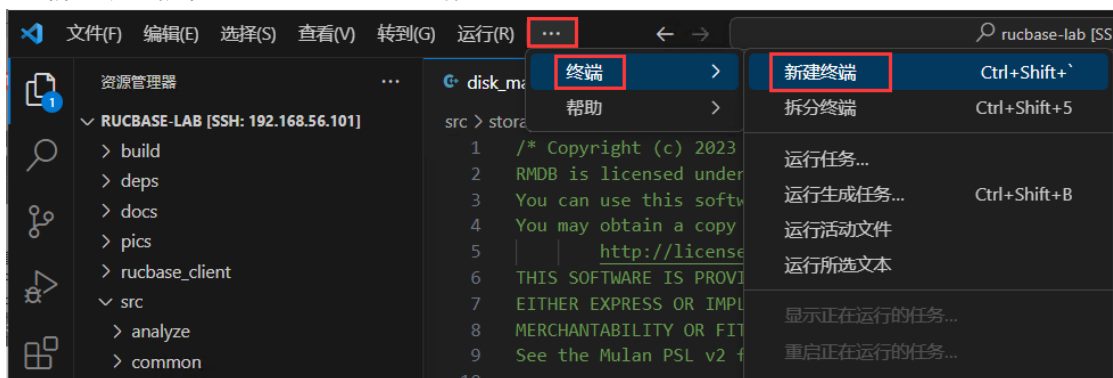
make disk_manager_test
./bin/disk_manager_test

make lru_replacer_test
./bin/lru_replacer_test

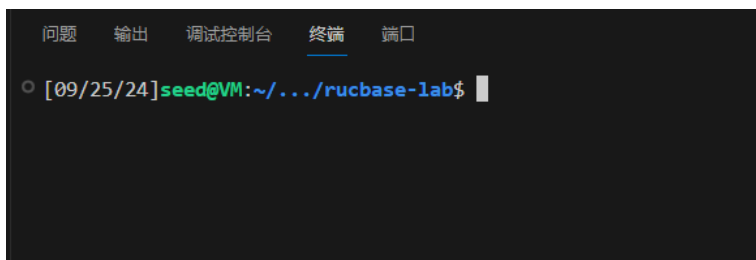
make buffer_pool_manager_test
./bin/buffer_pool_manager_test

make record_manager_test
./bin/record_manager_test
```

2. 我们可以直接在 VSCode 上打开终端执行：



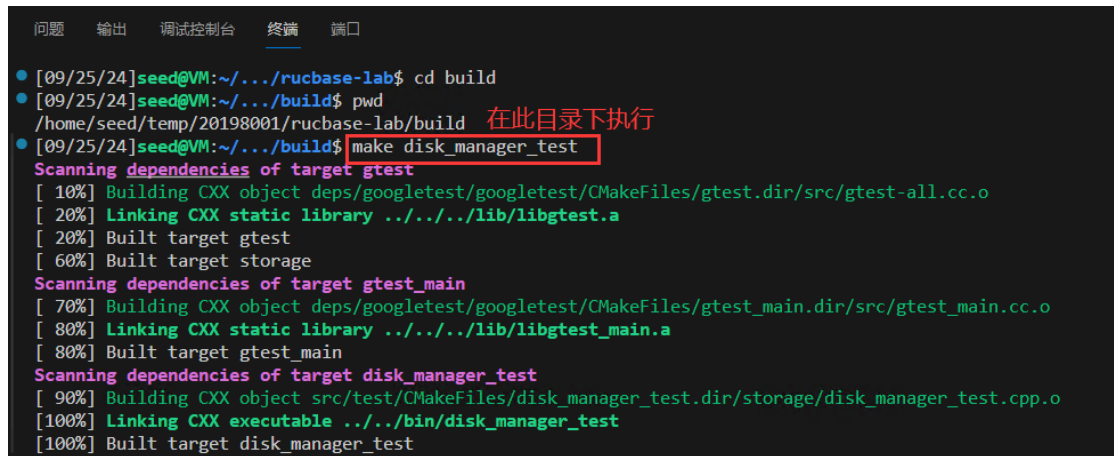
在 VSCode 上打开终端窗口：



3. 在 build 目录下执行 make disk_manager_test：

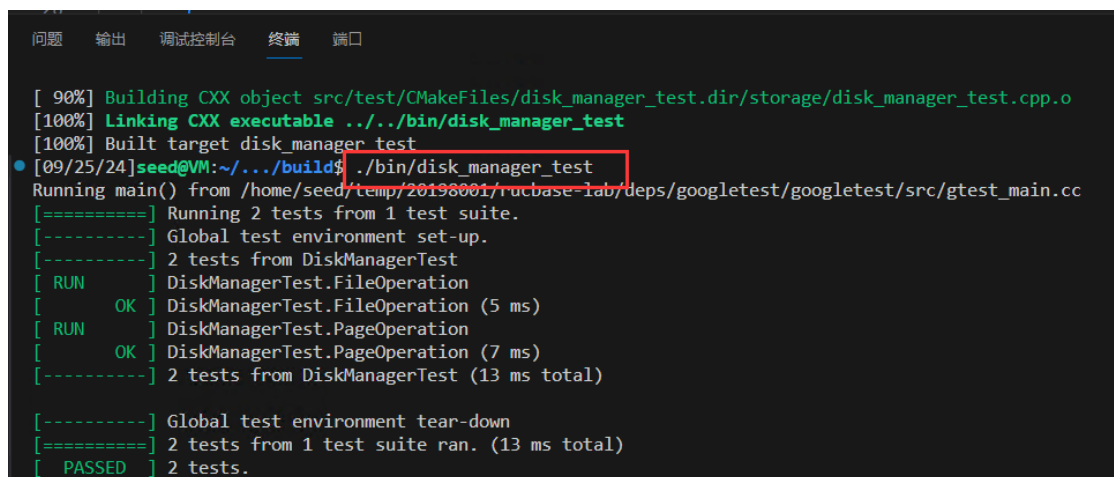
(Googletest 的细节可参考:

https://www.bilibili.com/video/BV14u411j7GV/?spm_id_from=333.788&vd_source=08013030e30f324d430dd1fbf06c5546)



```
问题 输出 调试控制台 终端 端口
[09/25/24]seed@VM:~/.../rucbase-lab$ cd build
[09/25/24]seed@VM:~/.../build$ pwd
/home/seed/temp/20198001/rucbase-lab/build 在此目录下执行
[09/25/24]seed@VM:~/.../build$ make disk_manager_test
Scanning dependencies of target gtest
[ 10%] Building CXX object deps/googletest/googletest/CMakeFiles/gtest.dir/src/gtest-all.cc.o
[ 20%] Linking CXX static library ../../lib/libgtest.a
[ 20%] Built target gtest
[ 60%] Built target storage
Scanning dependencies of target gtest_main
[ 70%] Building CXX object deps/googletest/googletest/CMakeFiles/gtest_main.dir/src/gtest_main.cc.o
[ 80%] Linking CXX static library ../../lib/libgtest_main.a
[ 80%] Built target gtest_main
Scanning dependencies of target disk_manager_test
[ 90%] Building CXX object src/test/CMakeFiles/disk_manager_test.dir/storage/disk_manager_test.cpp.o
[100%] Linking CXX executable ../../bin/disk_manager_test
[100%] Built target disk_manager_test
```

4. 执行测试:



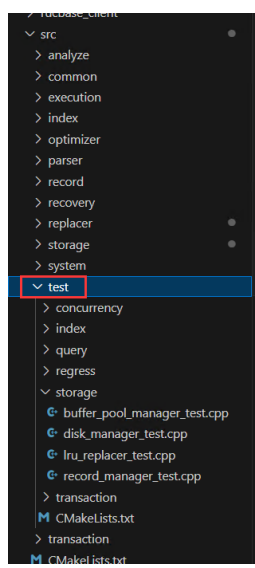
```
问题 输出 调试控制台 终端 端口
[ 90%] Building CXX object src/test/CMakeFiles/disk_manager_test.dir/storage/disk_manager_test.cpp.o
[100%] Linking CXX executable ../../bin/disk_manager_test
[100%] Built target disk_manager_test
[09/25/24]seed@VM:~/.../build$ ./bin/disk_manager_test
Running main() from /home/seed/temp/20198001/rucbase-lab/deps/googletest/googletest/src/gtest_main.cc
[=====] Running 2 tests from 1 test suite.
[-----] Global test environment set-up.
[-----] 2 tests from DiskManagerTest
[ RUN      ] DiskManagerTest.FileOperation
[ OK       ] DiskManagerTest.FileOperation (5 ms)
[ RUN      ] DiskManagerTest.PageOperation
[ OK       ] DiskManagerTest.PageOperation (7 ms)
[-----] 2 tests from DiskManagerTest (13 ms total)

[-----] Global test environment tear-down
[=====] 2 tests from 1 test suite ran. (13 ms total)
[ PASSED  ] 2 tests.
```

我们可以看到 disk_manager_test 中有 2 个测试用例, 均成功通过。

注意: 代码中 src\test 目录下存放的是测试代码, 类似于在面向对象课程中学过的 Junit 测试。当测试不通过时, 同学们可以打开相应的测试代码, 查看测试流程, 找到问题并修改。

每次修改代码后, 均需要保存代码, 然后用 make 命令编译, 最后运行测试 (参考上面 2 个图)。



5. 实验评分

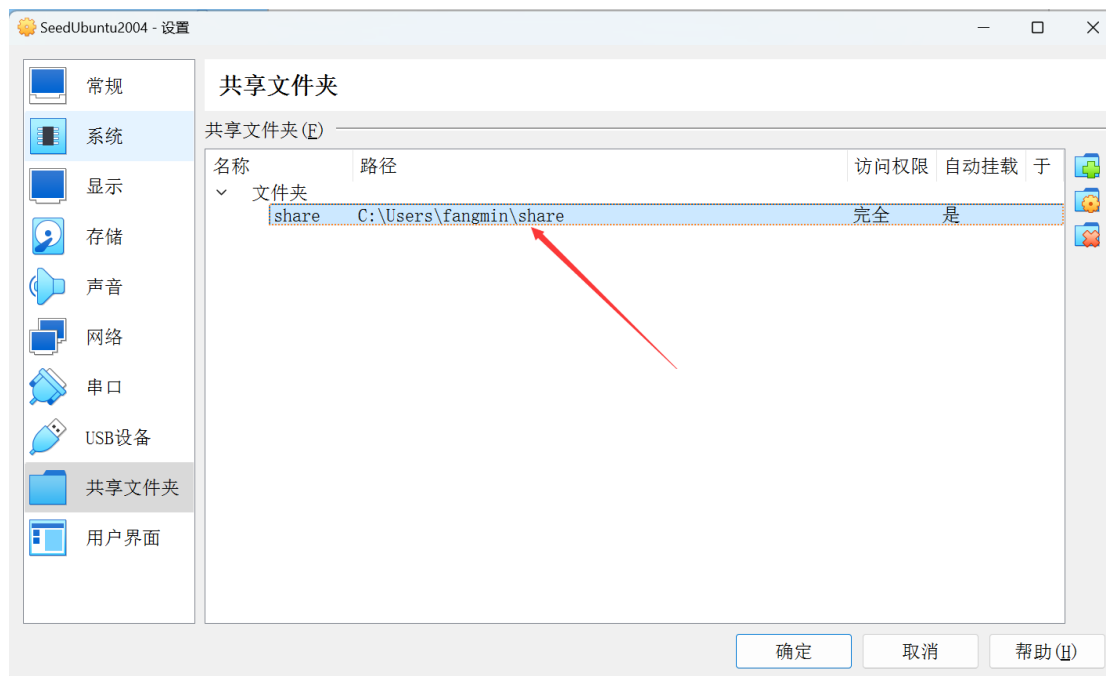
在本实验中，每个任务对应一个单元测试文件，每个测试文件中包含若干测试点。通过测试点即可得分，满分为 100 分。

测试文件及测试点如下：

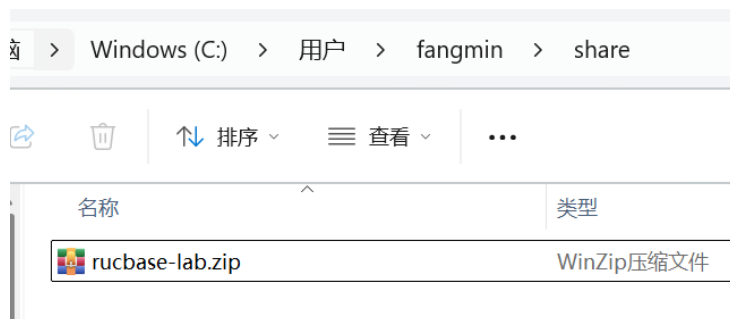
任务点	测试文件	分值
任务1.1 磁盘存储管理器	src/test/storage/disk_manager_test.cpp	10
任务1.2 缓冲池替换策略	src/test/storage/lru_replacer_test.cpp	20
任务1.3 缓冲池管理器	src/test/storage/buffer_pool_manager_test.cpp	40
任务2 记录管理器	src/test/storage/record_manager_test.cpp	30

附录 1 上传下载文件

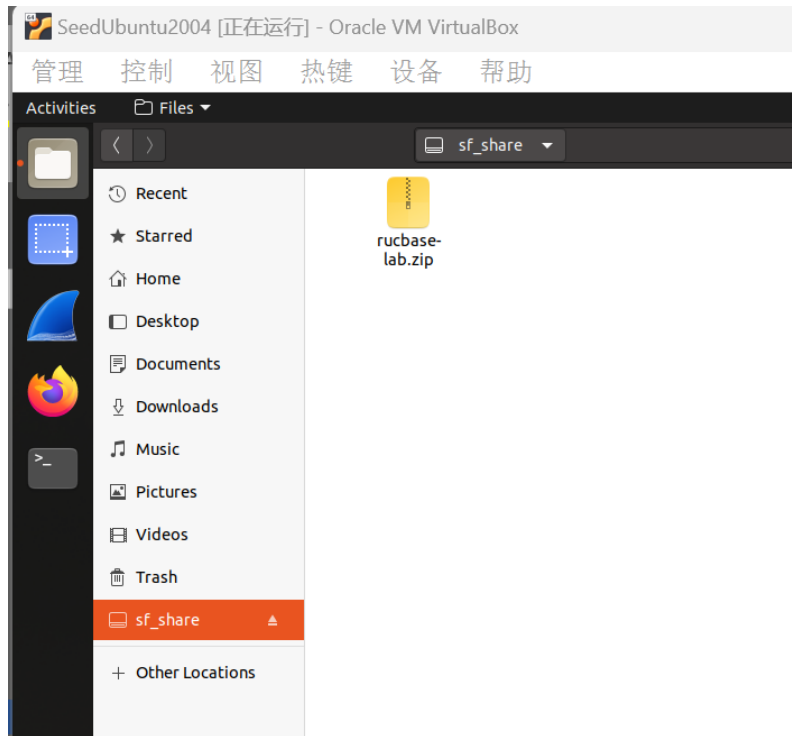
本机与虚拟机的文件交互可以用之前设置的共享文件夹：



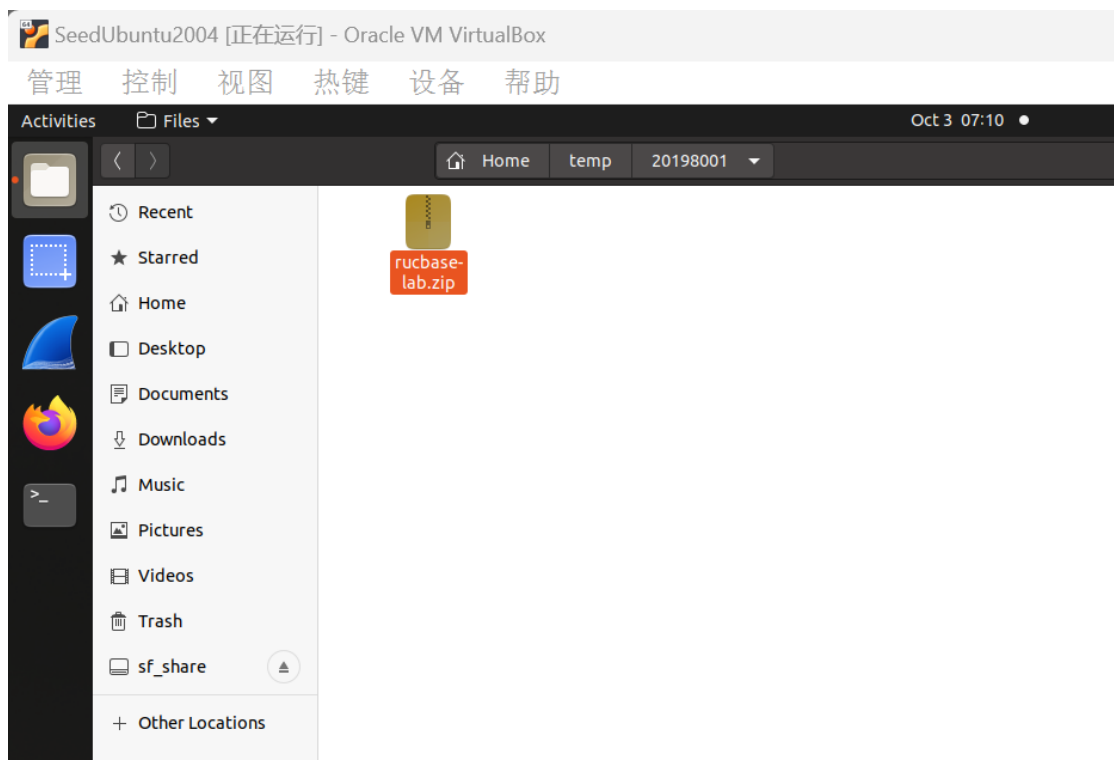
1、将代码压缩包放到共享文件夹：



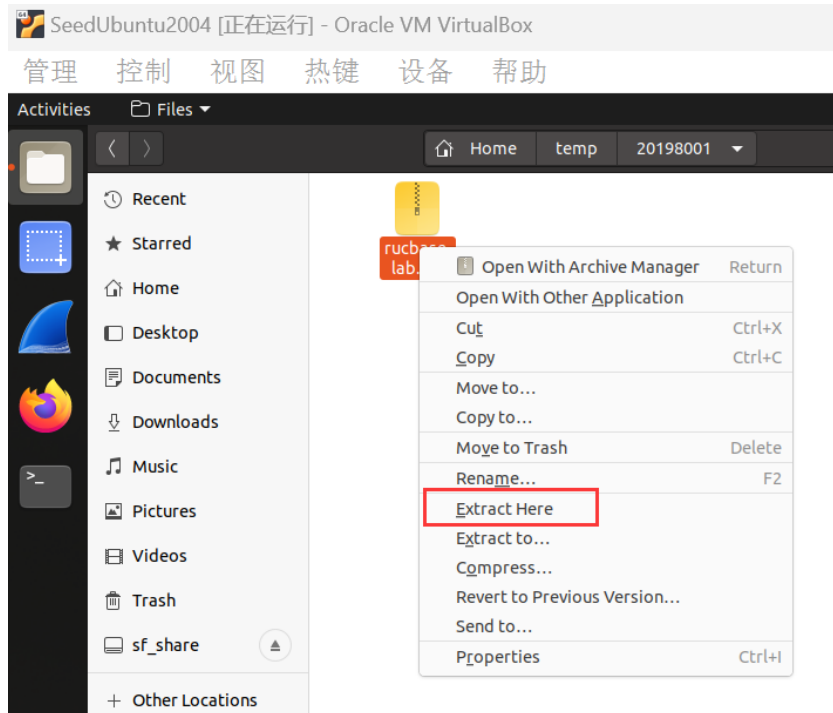
2、在虚拟机的 sf_share 目录下找到该压缩包：



3、拷贝到 temp 到你建的代码目录：



4、解压：



解压后可见代码：

